

AI DIGITAL OFFICE OS v1.4 Consolidated Master

Integrity-Patched Master Enterprise Blueprint, Full Data Model (41 Tables), Full API Contract (40 Endpoints), Closure Diagrams, Evidence-Disciplined Acceptance Checklist — Implementation-Lock Candidate

Consolidated Complete Package — Fully product-independent, standalone system

Finalized 2026-08-30

[Package Manifest — v1.4](#)

[Section A — README \(v1.4\)](#)

[AI DIGITAL OFFICE OS v1.4 — COMPLETE PACKAGE \(Integrity-Patched, Implementation-Lock Candidate\)](#)

[Main deliverables](#)

[Baseline](#)

[Build order](#)

[Important — read this before treating v1.4 as “final”](#)

[Section B — Master Enterprise Blueprint v1.4 \(Full Text, Clauses 1-76\)](#)

[AI DIGITAL OFFICE OS](#)

[Autonomous Agentic Software Factory & Organization Operating System](#)

[MASTER ENTERPRISE BLUEPRINT v1.4](#)

[0. Document Control](#)

[1. Executive System Definition](#)

[2. Approved v1.1 Architecture to Preserve](#)

[3. Executive Command Hierarchy](#)

[4. Agent Factory](#)

[5. Agent Lifecycle & Safety](#)

[6. Agent Registry Technical Contract](#)

[7. AI Provider Gateway](#)

[8. Provider Registry](#)

[9. Model Registry & Capability System](#)

[10. Intelligent Model Router](#)

[11. Model Evaluation Engine](#)

[12. Central Policy Engine](#)

[13. RBAC & Least Privilege](#)

[14. Approval Engine & Risk Tiers](#)

[15. Data Governance](#)

[16. Secrets Management](#)

[17. Task Engine](#)

[18. Durable Workflow Engine](#)

[19. Agent-to-Agent Communication](#)

[20. Dependency DAG](#)

[21. Prompt Registry](#)

[22. Knowledge & Memory](#)

[23. Artifact Lineage](#)

[24. Decision Records](#)

[25. Git & Engineering Control](#)

[26. Software Delivery Pipeline](#)

[27. Deployment Strategies](#)

[28. Disaster Recovery](#)

[29. Observability](#)

[30. Cost Governance](#)

[31. Error Handling & Reliability](#)

[32. Controlled Self-Healing](#)

[33. Project Factory](#)

[34. Domain Project Instances — Fully Decoupled, Product-Agnostic \(Revised in v1.4\)](#)

[35. Multi-Tenancy Readiness](#)

[36. Configuration & Feature Flags](#)

[37. Architecture Consistency Rules](#)

[38. Implementation Phases](#)

[39. Acceptance Test Matrix](#)

[40. Final Enterprise Principle](#)

[NEW IN v1.3 — Clauses 41 through 58 \(Gap Closure\)](#)

[41. Identity & Multi-Tenancy Model](#)

[42. Agent-to-Agent Message Bus \(Data Model Closure\)](#)

[43. Durable Workflow Engine \(Data Model Closure\)](#)

[44. RBAC — Full Data Model Closure](#)

[45. Agent Activation Authority \(Closes the Factory/Activation Ambiguity\)](#)

[46. Secrets Management — Full Data Model Closure](#)

[47. Knowledge & Memory — Full Data Model Closure](#)

[48. Artifact Registry — Full Data Model Closure](#)

[49. Error Handling — Concrete Retry & Backoff Numbers](#)

[50. Disaster Recovery — Concrete RPO/RTO Targets](#)

[51. Configuration & Feature Flags — Full Data Model Closure](#)

[52. API Versioning & Security \(Control-Plane Contract Hardening\)](#)

[53. Acceptance Test Matrix — Measurable Version](#)

[54. Interoperability Standards](#)

[55. Observability — Agent-Specific Trace Requirements](#)

[56. Indexing & Performance Baseline](#)

[57. Compliance & Audit Retention](#)

[58. Explicit Non-Goals \(Honesty Clause\)](#)

[NEW IN v1.4 — Clauses 59 through 76 \(Integrity Closure\)](#)

[59. Project Registry \(Closes CRITICAL finding — was missing entirely\)](#)

[60. Agent Versioning — Immutable, Activatable by Version \(Closes CRITICAL finding\)](#)

[61. Agent Run & Model Run Lineage \(Closes CRITICAL finding\)](#)

[62. Full Tenant Row-Level Security \(Closes CRITICAL finding\)](#)

[63. Agent Message Idempotency \(Closes HIGH finding\)](#)

[64. Model Evaluation History \(Closes HIGH finding\)](#)

[65. Cost & Usage Ledger \(Closes HIGH finding\)](#)

[66. Deployment & Release Registry \(Closes HIGH finding\)](#)

[67. MCP Tool Registry & Agent-Tool Bindings \(Closes HIGH finding\)](#)

[68. Complete API Lifecycle Coverage \(Closes HIGH finding\)](#)

[69. Schema / API / Domain-Model Consistency Gate \(Closes HIGH finding\)](#)

[70. Authentication Contract Detail \(Closes MEDIUM finding\)](#)

[71. Session Management Boundary \(Closes MEDIUM finding\)](#)

[72. Policy & Routing Decision Records \(Closes MEDIUM finding\)](#)

[73. Evidence Discipline — TARGET / TEST REQUIRED / VERIFIED \(Closes MEDIUM finding\)](#)

[74. Specification vs. Implementation Package Boundary \(Closes MEDIUM finding\)](#)

[75. Product Independence — Decoupling from Any Named Downstream Project \(User-Directed Change\)](#)

[76. Implementation-Lock Gate](#)

[Section C — Database Schema v1.4 \(Full SQL — 41 Tables, Full RLS\)](#)

[Section D — Control Plane OpenAPI v1.4 \(Full YAML — 40 Endpoints\)](#)

[Section E — Architecture & Sequence Diagrams v1.4](#)

[AI DIGITAL OFFICE OS v1.4 Architecture Diagrams](#)

[AI DIGITAL OFFICE OS v1.2/v1.3/v1.4 Architecture \(unchanged since v1.2, still authoritative\)](#)

[Agent Factory lifecycle \(unchanged from v1.2\)](#)

[Software Factory lifecycle \(unchanged from v1.2\)](#)

[NEW IN v1.3 — Sequence & Structure Diagrams](#)

[1. Task Routing Sequence \(clauses 10, 17, 43, 49\)](#)

[2. Agent Creation → Activation Sequence \(clauses 4, 5, 14, 45\)](#)

[3. Approval / Risk-Tier Escalation Flow \(clause 14\)](#)

[4. Multi-Tenant Isolation \(structural, clause 41/44\)](#)

[NEW IN v1.4 — Two More Closure Diagrams](#)

[5. Full Artifact Lineage Resolution \(clauses 23, 59, 60, 61, 66 — closes CRITICAL “undefined run entities” finding\)](#)

[6. Authentication & Tenant Scoping Sequence \(clauses 70, 71 — closes MEDIUM auth-contract finding\)](#)

[Section F — Agent Specification Template v1.4](#)

[Agent Specification Template v1.4](#)

[## Capabilities](#)

[## Required Model Capabilities](#)

[Tools \(declared via MCP — Model Context Protocol, blueprint clause 54\)](#)

[## Permissions](#)

[Data Classification Allowed](#)

[Input Contract](#)

[Output Contract](#)

[Security Level](#)

[Sandbox Policy](#)

[Idempotency](#)

[Evaluation Suite](#)

[Lifecycle](#)

[Tool Bindings \(v1.4, blueprint clause 67\)](#)

[External Interoperability \(optional, disabled by default\)](#)

[Owner / Governance](#)

[Section G — Non-Functional Requirements & SLOs \(All Items Labeled TARGET\)](#)

[AI DIGITAL OFFICE OS v1.4 — Non-Functional Requirements & Service Level Objectives](#)

[Availability](#)

[Latency](#)

[Reliability — Retry & Backoff \(blueprint clause 49\)](#)

[Scalability Targets \(initial baseline, re-test each phase\)](#)

[Disaster Recovery \(blueprint clause 50\)](#)

[Security](#)

[Cost Governance](#)

[Observability](#)

[Self-Healing Bound](#)

[Section H — Implementation Acceptance Checklist v1.4 \(Evidence-Disciplined\)](#)

[AI DIGITAL OFFICE OS v1.4 Implementation Acceptance Checklist](#)

[Evidence-Disciplined Version \(blueprint clause 73\)](#)

[Architecture — \[TARGET\]](#)

[Project Registry — \[TARGET\] \(new in v1.4, closes CRITICAL finding\)](#)

[Agent Versioning — \[TARGET\] \(new in v1.4, closes CRITICAL finding\)](#)

[Lineage — \[TARGET\] \(new in v1.4, closes CRITICAL finding\)](#)

[Multi-Tenancy & RLS — \[TARGET\] \(full coverage new in v1.4, closes CRITICAL finding\)](#)

[Agent Messaging — \[TARGET\] \(idempotency added in v1.4, closes HIGH finding\)](#)

[AI Platform — \[TARGET\]](#)

[Cost & Usage — \[TARGET\] \(new in v1.4, closes HIGH finding\)](#)

[Deployment & Release — \[TARGET\] \(new in v1.4, closes HIGH finding\)](#)

[MCP Tool Registry — \[TARGET\] \(new in v1.4, closes HIGH finding\)](#)

[API Completeness — \[TARGET\] \(expanded in v1.4, closes HIGH finding\)](#)

[Schema/API Consistency — \[TARGET\] \(new in v1.4, closes HIGH finding\)](#)

[Authentication & Session Boundary — \[TARGET\] \(detailed in v1.4, closes MEDIUM finding\)](#)

[Security — \[TARGET\]](#)

[Engineering — \[TARGET\]](#)

[Memory & Artifacts — \[TARGET\]](#)

[Operations — \[TARGET\]](#)

[Extensibility — \[TARGET\]](#)

[Product Independence — \[TARGET\] \(new in v1.4, user-directed change\)](#)

[Evidence Discipline — \[TARGET\] \(new in v1.4, closes MEDIUM finding — this rule governs this whole file\)](#)

[Implementation-Lock Gate — \[TARGET\] \(blueprint clause 76\)](#)

[Section I — Changelog: v1.3 → v1.4 \(Finding-by-Finding Closure Map\)](#)

[Changelog — v1.3 → v1.4](#)

[Why this version exists](#)

[CRITICAL findings — closed](#)

[HIGH findings — closed](#)

[MEDIUM findings — closed](#)

[Also in this version — user-directed product decoupling](#)

[What did NOT change](#)

[Explicitly NOT done \(honesty — clause 74\)](#)

[Section J — Baseline & Final Honesty Statement](#)

Package Manifest — v1.4

#	File	Section
1	README.md	A
2	AI_DIGITAL_OFFICE_OS_MASTER_BLUEPRINT_v1.4.md (76 clauses)	B
3	database_schema_v1.4.sql (41 tables, full RLS)	C
4	control_plane_openapi_v1.4.yaml (40 endpoints)	D
5	architecture_diagrams_v1.4.md (3 original + 4 v1.3 + 2 new v1.4 diagrams)	E
6	agent_specification_template_v1.4.md	F
7	nfr_slo_v1.4.md (all items labeled TARGET)	G
8	implementation_acceptance_checklist_v1.4.md (TARGET/TEST REQUIRED/VERIFIED)	H
9	CHANGELOG_v1.3_to_v1.4.md (finding-by-finding closure map)	I
10	v1.3_baseline/ (full v1.3 package, preserved)	Noted in J

Section A — README (v1.4)

AI DIGITAL OFFICE OS v1.4 — COMPLETE PACKAGE (Integrity-Patched, Implementation-Lock Candidate)

This package is the v1.4 update to the v1.3 baseline. It exists specifically to close every structural closure issue identified in the independent **AI DIGITAL OFFICE OS v1.4 Technical Integrity Review** (dated 2026-08-30) of the v1.3 Consolidated Master: missing Project Registry, incomplete agent versioning, unresolvable artifact/run lineage, partial Row-Level Security, missing message idempotency, missing model evaluation history, missing cost/usage ledger, missing deployment/release records, missing MCP tool registry, incomplete API lifecycle coverage, schema/API drift risk, a thin auth contract, an unmodeled session boundary, missing policy/routing decision records, and acceptance evidence that did not distinguish target claims from verified ones.

v1.4 is an integrity patch, not a second architecture — the review’s own words. Nothing about the approved hierarchy, Agent Factory, capability-based model routing, multi-provider gateway, Policy Engine, GREEN/YELLOW/RED governance, durable workflows, tiered memory, artifact immutability, independent QA, bounded self-healing, or MCP/A2A direction has changed.

Also in v1.4, on explicit user instruction: this Office is now formally decoupled from any specific named downstream product. The one clause (34) and one Layer-5 mention that previously named a specific project as a privileged “first implementation” have been rewritten to be fully product-agnostic — see the blueprint’s clause 34 and clause 75, and `CHANGELOG_v1.3_to_v1.4.md` for the exact wording change.

Main deliverables

- `AI_DIGITAL_OFFICE_OS_MASTER_BLUEPRINT_v1.4.md` — full spec, clauses 1–76 (1–58 preserved with one deliberate, documented edit to clause 34; 59–76 new)
- `database_schema_v1.4.sql` — full PostgreSQL logical schema (41 tables: the 27 from v1.3 plus 14 closing the integrity findings; full RLS with USING + WITH CHECK on every tenant-scoped table)
- `control_plane_openapi_v1.4.yaml` — full OpenAPI 3.0.3 contract (40 endpoints, detailed JWT auth contract, 29 component schemas, mandatory Idempotency-Key header)
- `architecture_diagrams_v1.4.md` — 3 original + 4 v1.3 sequence diagrams + 2 new v1.4 closure diagrams (full lineage resolution, auth/tenant-scoping sequence)
- `agent_specification_template_v1.4.md` — updated for version-targeted activation and stable tool-ID bindings
- `nfr_slo_v1.4.md` — same v1.3 numbers, now explicitly labeled TARGET per the evidence-discipline rule
- `implementation_acceptance_checklist_v1.4.md` — every item now labeled TARGET / TEST REQUIRED / VERIFIED, not a bare checkbox
- `CHANGELOG_v1.3_to_v1.4.md` — honest, itemized diff mapped directly to each review finding
- `v1.3_baseline/` — the full v1.3 package preserved, not removed

Baseline

The v1.3 package (including its consolidated PDF) remains the architectural baseline and is not contradicted — v1.4 only closes the structural gaps the independent review found between v1.3’s narrative and its schema/API.

Build order

Blueprint (clauses 1–76) → database_schema_v1.4.sql → control_plane_openapi_v1.4.yaml → architecture_diagrams_v1.4.md → agent_specification_template_v1.4.md → nfr_slo_v1.4.md → implementation_acceptance_checklist_v1.4.md.

Important — read this before treating v1.4 as “final”

Per the review’s own Implementation-Lock Gate (blueprint clause 76), this specification is an **implementation-lock candidate**, not yet locked. It becomes lockable only once the SQL/API/domain-model consistency check, full RLS adversarial test, end-to-end lineage resolution, and version-targeted activation are all VERIFIED — not merely present in the schema. This remains a specification and data-model package, not running code; no infrastructure-as-code, cloud topology, real provider contracts, penetration-test results, or production credentials are included or fabricated (clause 74).

Section B — Master Enterprise Blueprint v1.4 (Full Text, Clauses 1-76)

AI DIGITAL OFFICE OS

Autonomous Agentic Software Factory & Organization Operating System

MASTER ENTERPRISE BLUEPRINT v1.4

ENTERPRISE TECHNICAL & IMPLEMENTATION SPECIFICATION — INTEGRITY-PATCHED, IMPLEMENTATION-LOCK CANDIDATE

0. Document Control

- Baseline: AI DIGITAL OFFICE OS Master Enterprise Blueprint v1.3 (itself baselined on v1.2 and v1.1).
- Target: Version 1.4 — an **integrity patch, not a second architecture**. It closes every structural closure issue identified in the independent v1.4 Technical Integrity Review of the v1.3 Consolidated Master (dated 2026-08-30): missing Project Registry, incomplete agent versioning, unresolvable artifact/run lineage, partial RLS, missing message idempotency, missing model evaluation history, missing cost/usage ledger, missing deployment/release records, missing MCP tool registry, incomplete API lifecycle coverage, schema/API drift risk, thin auth contract, unmodeled session boundary, missing policy/routing decision records, and acceptance evidence that did not distinguish target from verified.
- Also in v1.4, on explicit user instruction: clause 34 (and the one Layer-5 mention that referenced it) has been decoupled from any specific named downstream product. AI DIGITAL OFFICE OS is now stated as fully product-agnostic and independent — see clause 34 and clause 75.
- Status: This is the specification the independent review named as the **implementation-lock candidate**, contingent on the closure gate in clause 76. It is still a specification and data-model package, not running production code — no infrastructure, provider contracts, penetration-test results, or credentials are included or fabricated (unchanged principle from v1.2/v1.3, reaffirmed in clause 74).
- Scope: clauses 1–40 (v1.2 baseline) and 41–58 (v1.3 additions) are preserved below, with exactly one deliberate content edit — clause 34 — made on explicit user instruction and documented in the changelog; every other clause is unchanged. New clauses 59–76 close the v1.4 integrity findings.
- Core principles (unchanged from v1.3): Model-Agnostic, Provider-Agnostic, Agent-Extensible, Tool-Extensible, Secure, Auditable, Scalable, Recoverable, Testable, Tenant-Isolated, Interoperable (MCP/A2A), Measurably Reliable. **Added for v1.4:** Lineage-Complete (every artifact, run, and decision resolves end-to-end through the relational model), Evidence-Disciplined (no acceptance item may be marked VERIFIED without an executed test producing evidence), Product-Independent (zero hardcoded coupling to any specific downstream project).

1. Executive System Definition

- AI DIGITAL OFFICE OS is a governed AI organization and autonomous software factory. It receives a business objective, decomposes it into governed work, assigns specialist agents, selects appropriate AI capabilities, invokes controlled tools, produces versioned artifacts, verifies outputs independently, and releases approved results.
- The system is not a single chatbot and not a collection of disconnected agents. It is an operating system for coordinated AI work with hierarchy, durable workflows, policies, memory, artifacts, security, quality gates, observability, and human authority.
- Authoritative business calculations and invariants remain in deterministic domain services. AI decides and orchestrates; deterministic engines calculate and enforce business rules.

2. Approved v1.1 Architecture to Preserve

- Layer 1: Owner & Human Governance.
- Layer 2: Executive Management: Master CEO, Executive Manager, Assistant Manager.
- Layer 3: Orchestration Control Plane: stateful workflows, dependency DAG, task dispatch, deadlock detection and run tracking.
- Layer 4: Agent Factory & Registry: specification, cloning/specialization, sandboxing, test generation, benchmarking, capability profiling and lifecycle.
- Layer 5: Specialized Agents: product, research, architecture, UI/UX, frontend, backend, QA, security, DevOps, prompt/creative and tenant-defined domain operational agents. **(v1.4: no product-specific agent set is hard-coded into the core architecture — see clause 75.)**
- Layer 6: AI Provider Gateway & Router: provider adapters, model registry, capability classes, intelligent routing and controlled credentials.
- Layer 7: Tool Gateway & Services: MCP, Git, shell, Docker, Postgres, browsers, web/search and automation services under RBAC.
- Layer 8: Memory, Artifacts & Audit: pgvector knowledge, project memory, immutable artifacts, QA, security scanning, telemetry and audit trails.
- **v1.3 addition — Layer 9: Identity & Tenancy:** tenants/organizations, users, tenant-scoped roles and permissions, session management — sits beneath Layer 1 as the isolation boundary every other layer must respect.

3. Executive Command Hierarchy

- Owner/Chairman is the final authority for strategy, RED operations, budget overrides, destructive production actions and emergency kill-switch control.
- Master CEO converts intent into goals, project constitutions, milestones and quality gates.
- Executive Manager performs work breakdown, departmental allocation and scheduling.
- Assistant Manager coordinates execution, dependencies, artifact verification and next-agent triggering.
- Departments execute typed tasks through governed agents. No department bypasses the orchestration and policy layers for critical work.

4. Agent Factory

- Agent Factory is a first-class subsystem responsible for creating, cloning, extending, specializing, evaluating, activating, upgrading, deprecating and retiring agents.
- Creation pipeline: Request → Requirement Analysis → Capability Design → Agent Specification → Prompt/Instruction Generation → Tool Selection → Permission Generation → Model Policy → Input/Output Schemas → Test Generation → Sandbox → Evaluation → Security → Approval → Registry → Activation.
- Generated agents must start in a sandbox and must not receive unrestricted production credentials, root access or unrestricted database access.
- Agent cloning and specialization preserve lineage through `parent_agent_id` and independent versions.
- **v1.3 addition:** the Agent Factory is itself a governed agent — it cannot activate an agent into production; it can only advance an agent to `APPROVED`. Activation is a separate, human- or policy-triggered transition (see clause 45).

5. Agent Lifecycle & Safety

- Lifecycle states: DRAFT, SANDBOX, TESTED, EVALUATED, APPROVED, ACTIVE, UPDATED, DEPRECATED, RETIRED.
- Promotion requires the applicable quality and security gates. An agent may be automatically created but production activation remains governed.
- Agent creation must be bounded. No agent may recursively create unlimited agents without Agent Factory, Policy Engine and approval controls.
- **v1.3 addition:** hard ceiling of 25 active agents per department and 200 active agents per tenant by default (configurable per-tenant policy, itself an approval-gated change) to prevent uncontrolled agent sprawl.

6. Agent Registry Technical Contract

- Required identity: `agent_id`, `name`, `department`, `role`, `purpose`, `version`, `lifecycle_state`.
- Capability contract: `capabilities`, `required_model_capabilities`, `preferred_provider`, `preferred_model`, `fallbacks`.
- Execution contract: `allowed_tools`, `permissions`, `data_access`, `sandbox_policy`, `input_schema`, `output_schema`.
- Operational contract: `evaluation_score`, `success_rate`, `average_latency`, `average_cost`, `status`, `created_at`, `updated_at`, `parent_agent_id`.
- Registry is authoritative for what an agent is allowed to do. It is not merely a directory.
- **v1.3 addition:** every agent record is `tenant_id`-scoped (see clause 41); a `NULL tenant_id` marks a system/global agent template that a tenant may clone into its own scope but never execute directly.

7. AI Provider Gateway

- All external AI access passes through a central AI Provider Gateway. Agents do not directly hold unrestricted provider API keys.
- Provider adapters isolate vendor-specific APIs from the core operating system. Initial adapters may include OpenAI/GPT, Anthropic/Claude, Google/Gemini, Moonshot/Kimi, OpenAI-compatible APIs and local model runtimes.
- Adding a future provider follows: Adapter → Registration → Capability Mapping → Health Probe → Security Review → Evaluation → Approval → Production.
- Provider outages, rate limits and transient errors must not halt the Office when an approved compatible fallback exists.

8. Provider Registry

- Provider entity fields: provider_id, provider_name, provider_type, adapter_type, protocol, endpoint, authentication_method, supported_capabilities, supported_models, rate_limits, quota_rules, pricing_rules, privacy_classification, data_residency, availability, health_status, version and timestamps.
- Provider identity is separate from model identity so models can change independently of provider integration.

9. Model Registry & Capability System

- Model registry stores model identity, provider relationship, version, context window, supported input/output types, tool calling, structured output, vision, coding, reasoning, research, latency, cost, privacy and health.
- Agents request capabilities rather than hard-coding model names. Example capability classes: HEAVY_CODING, DEEP_RESEARCH, REASONING, VISION_UI, LONG_CONTEXT, FAST_ROUTINE, LOW_COST, HIGH_RELIABILITY, STRUCTURED_OUTPUT, TOOL_USE, LOCAL_OFFLINE.
- Model Registry, pricing, health and evaluation records are independently maintainable.

10. Intelligent Model Router

- Routing inputs: task, agent, required capability, project policy, security classification, data classification, provider health, model health, quality, cost, latency, context requirements, tool requirements and owner override.
- Every routing decision is auditable: task_id, agent_id, requested capability, selected provider, selected model, reason, policy result and timestamp.
- Fallback chain: Primary → Approved Secondary → Approved Local Engine → Human Escalation. Fallback must preserve capability, privacy and security requirements.

11. Model Evaluation Engine

- Every production model should have standardized benchmarks covering coding, reasoning, research, tool use, structured output, vision, reliability, latency, cost efficiency, instruction following, security and privacy compliance.
- Evaluation results are versioned and may be used by the router. Models can be re-evaluated as providers change versions or pricing.

12. Central Policy Engine

- Policy Engine is a first-class service. It evaluates Agent + Task + Data + Tool + Model + Provider + Environment + Risk and returns ALLOW, DENY, REQUIRE_APPROVAL or REQUIRE_ESCALATION.
- Policies govern permissions, tool access, data access, model/provider selection, production operations, external API calls, sensitive information, deployment and destructive operations.
- Policies are machine-readable, versioned, auditable and applied before execution.

13. RBAC & Least Privilege

- Authorization chain: Agent → Role → Permission → Tool → Resource → Action.
- Support role, resource, action, environment and project scoped permissions. Typical actions include READ, WRITE, CREATE, UPDATE, DELETE, EXECUTE, DEPLOY, APPROVE, EXPORT and ADMINISTER.
- Default posture is deny. Permissions are explicitly granted and scoped.
- **v1.3 — full data-model closure, see clause 44.**

14. Approval Engine & Risk Tiers

- Approval flow: Request → Risk Classification → Policy Check → Approval Routing → Decision → Execution → Audit.
- GREEN permits autonomous actions within policy. YELLOW requires designated management review. RED requires Owner or explicitly authorized human sign-off.
- Approval records include request_id, requester, agent, action, risk, reason, approver, decision, timestamp, expiration and execution result.

15. Data Governance

- Data classifications: PUBLIC, INTERNAL, CONFIDENTIAL, SENSITIVE, RESTRICTED.
- Each class defines allowed agents, tools, models, providers, cloud/local rules, logging restrictions, export restrictions, retention and encryption requirements.
- Sensitive or restricted information must not be sent to an external model unless explicitly allowed by policy.

16. Secrets Management

- Use a dedicated Secrets Management Service for AI keys, database credentials, infrastructure credentials, OAuth secrets, signing keys and deployment credentials.
- Agents never receive unrestricted raw secrets. Use scoped and preferably short-lived credentials. Every secret access is auditable.
- **v1.3 — full data-model closure, see clause 46.**

17. Task Engine

- Every task has task_id, project_id, workflow_id, parent_task_id, assigned_agent, required_capability, priority, risk, security_level, dependencies, input, expected_output, status, retry_count, deadline and timestamps.
- Task states: CREATED, QUEUED, ASSIGNED, RUNNING, WAITING, BLOCKED, FAILED, RETRYING, COMPLETED, CANCELLED, ESCALATED.

18. Durable Workflow Engine

- Every critical workflow supports start, pause, resume, retry, cancel, escalate, rollback and completion.
- Workflow state must survive process restarts. Critical workflows must not depend solely on volatile in-memory state.
- **v1.3 — full data-model closure, see clause 43.** Workflow execution follows durable-execution semantics (event-sourced history, replayable state, per-activity retry policy) consistent with current production practice for long-running agent workflows.

19. Agent-to-Agent Communication

- Critical agent communication uses typed messages rather than uncontrolled conversational state.
- Message contract: task_id, workflow_id, sender_agent, receiver_agent, message_type, purpose, priority, security_level, input_schema, expected_output, dependencies, deadline, artifact_reference, status, result and error.
- **v1.3 — full data-model closure, see clause 42.** Cross-boundary agent communication (to external/partner agent systems) additionally follows an A2A-style contract: HTTP+JSON-RPC transport, agent capability cards, and task lifecycle negotiation — kept optional and disabled by default until an explicit integration is approved.

20. Dependency DAG

- Typical chain: Research → Product → Requirements → Architecture → UI/UX → Development → Review → QA → Security → Deployment → Monitoring.
- DAG engine detects circular dependencies, deadlocks, missing dependencies, failed dependencies, blocked tasks and timeout conditions.

21. Prompt Registry

- Prompt Registry stores prompt_id, agent_id, version, system instructions, variables, input/output contracts, compatible capabilities, evaluation score, security classification, author, timestamps, changelog and rollback version.
- Prompt changes are versioned and must pass regression evaluation before production use.

22. Knowledge & Memory

- Memory classes: working memory, task memory, agent memory, project memory, organizational memory, decision memory and artifact memory.
- Retrieval flow: Query → Authorization → Retrieval → Filtering → Ranking → Context Assembly → Agent → Source/Citation Reference.
- Memory ownership, retention, access, versioning and deletion are policy controlled.
- **v1.3 — full data-model closure, see clause 47.** Retrieval is tiered: hot/working memory in a fast key-value layer, durable structured memory in relational tables, and semantic memory in pgvector with HNSW indexing — routed by access pattern rather than sending every lookup through vector search.

23. Artifact Lineage

- Full lineage: Owner Command → Project → Task → Agent Run → Model Run → Tool Calls → Artifact → Review → Approval → Git Commit → Build → Deployment → Release.
- Important artifacts must be immutable/versioned and traceable to their originating workflow and decision context.

- **v1.3 — full data-model closure, see clause 48.**

24. Decision Records

- Maintain Architecture/Decision Records for important choices. Store decision_id, project_id, decision, reason, alternatives, evidence, agent, model, author, approval, timestamp, impact and status.

25. Git & Engineering Control

- Git is a controlled subsystem supporting branches, commits, pull requests, code review, tags, releases and rollback.
- Production branches are protected. Agents operate with scoped repository permissions. Production deployment cannot originate from unreviewed changes.

26. Software Delivery Pipeline

- Development → Unit Test → Integration Test → Static Analysis → Security Scan → Code Review → QA → Staging → Approval → Production → Health Check → Monitoring.
- Production failures support policy-controlled automated rollback.

27. Deployment Strategies

- Support standard, rolling, blue/green and canary deployment strategies.
- Every deployment has version, health checks, readiness checks, approval state, audit trail and rollback trigger.

28. Disaster Recovery

- Define database, artifact, configuration and backup policies, point-in-time recovery, restore testing, RPO and RTO.
- Recovery procedures must be tested periodically and their results recorded.
- **v1.3 — concrete targets, see clause 50.**

29. Observability

- Unified logs, metrics, traces, events and alerts cover agents, tasks, workflows, models, providers, tools, APIs, databases, deployments, costs and errors.
- Every important request carries a correlation ID for end-to-end tracing.
- **v1.3 addition:** agentic workflows require observability beyond standard APM — non-deterministic agent decision paths (which model was chosen, why, what the agent considered and rejected) must be captured as structured trace events, not just latency/error metrics.

30. Cost Governance

- Track cost per request, task, agent, project and provider, plus daily and monthly budgets.
- Support warning, soft limit, hard limit and Owner approval. Automatic low-cost routing is allowed only when policy and quality requirements remain satisfied.

31. Error Handling & Reliability

- Standard error classes: VALIDATION_ERROR, AUTHORIZATION_ERROR, POLICY_ERROR, TOOL_ERROR, MODEL_ERROR, PROVIDER_ERROR, NETWORK_ERROR, TIMEOUT, DATABASE_ERROR, ARTIFACT_ERROR, DEPLOYMENT_ERROR, UNKNOWN_ERROR.
- Each error records error_id, task/workflow/agent, service, severity, message, retryability, retry count, recovery action and timestamp.
- Critical operations are idempotent. Retries use bounded exponential backoff and dead-letter handling where appropriate.
- **v1.3 — concrete retry/backoff numbers, see clause 49.**

32. Controlled Self-Healing

- Existing bounded self-healing remains: Failure → Root Cause Analysis → Patch Agent → Regression Verification.
- Maximum automatic attempts remain strictly bounded at three before human escalation. Self-healing must not bypass governance or create uncontrolled duplicate production actions.

33. Project Factory

- Universal lifecycle: IDEA → DISCOVERY → RESEARCH → REQUIREMENTS → PRODUCT PLAN → ARCHITECTURE → UX/UI → DESIGN → DEVELOPMENT → CODE REVIEW → QA → SECURITY → STAGING → APPROVAL → DEPLOYMENT → MONITORING

→ MAINTENANCE.

- Supported product types include websites, web applications, mobile applications, APIs, SaaS, dashboards, POS, CRM, ERP, internal tools, automation systems, AI applications and future software categories.

34. Domain Project Instances — Fully Decoupled, Product-Agnostic (Revised in v1.4)

- **v1.4 change, made on explicit user instruction:** all prior references to any specific named downstream product (including the previously named POS example) as a privileged, “first,” or architecturally locked factory implementation have been removed from this clause and from Layer 5. AI DIGITAL OFFICE OS is a completely independent, general-purpose factory. It has zero hardcoded coupling to any specific downstream product, and no product is treated as special-cased or pre-selected by the architecture.
- Any downstream system a tenant wants to build — a POS, a CRM, an ERP, a website, or any other product type — is created the same way as every other project: as an ordinary row in `project_registry` (clause 59), run through the standard Project Factory (clause 33), with agents drawn from the same general-purpose Agent Factory. No project type receives special-cased agents, tables, or bypass paths.
- Which product (if any) a tenant chooses to build first, second, or ever, is entirely the tenant’s own decision, made at the project level, and carries no architectural weight or precedent for the core Office.
- This clause supersedes the v1.2/v1.3 wording that named a specific POS project as a locked example; that coupling is intentionally and permanently removed.

35. Multi-Tenancy Readiness

- Even if the first deployment is single-organization, the data model and authorization boundaries should be ready for organizations, teams, users, roles and isolated projects without a complete core rewrite.
- **v1.3 — this is no longer aspirational. Multi-tenancy is implemented in the v1.3 schema from day one; see clause 41.**

36. Configuration & Feature Flags

- Operational configuration is versioned, environment-specific, auditable, validated and rollback-capable.
- Feature flags support staged activation of agents, models, providers, tools, workflows and experimental capabilities.
- **v1.3 — full data-model closure, see clause 51.**

37. Architecture Consistency Rules

- Every agent has governance, permissions and evaluation. Every permission maps to policy. Every model belongs to a provider. Every provider has an adapter. Every task belongs to a workflow/project. Every critical action has a risk level and approval path. Every artifact has lineage. Every production deployment has rollback. Every secret has controlled access. Every important event is observable.
- **v1.3 addition:** every tenant-scoped table carries `tenant_id`; every junction table carries `tenant_id` alongside its foreign keys so referential integrity alone cannot produce a cross-tenant privilege escalation (composite-key enforcement, not just application-layer checks).

38. Implementation Phases

- Phase 0: Constitution and governance.
- Phase 1: Core infrastructure and control plane.
- Phase 2: Agent Registry, Factory, sandbox and evaluation.
- Phase 3: MCP and Tool Gateway.
- Phase 4: Provider Gateway, Provider Registry, Model Registry and Router.
- Phase 5: Core engineering departments.
- Phase 6: Knowledge, memory, artifacts and telemetry.
- Phase 7: Advanced autonomy, cloning, specialization and bounded self-healing.
- Phase 8: Full Project Factory and first production project launch — product-agnostic; the choice of which product to launch first is a tenant decision, not an architectural one (see clause 34, revised in v1.4).
- Phase 9+: Future scale, multi-tenancy, advanced deployment and provider/model expansion.
- **v1.3 note:** because multi-tenancy, RBAC, memory, artifacts, messaging and workflow durability are now in the Phase 1–2 schema instead of deferred to Phase 9+, the phase plan is unchanged in name but Phase 1 is now heavier — this is intentional and prevents the rewrite risk flagged in the v1.2 review.

39. Acceptance Test Matrix

- Architecture: all v1.1 components preserved and connected.
- Agents: creation, sandbox, evaluation, approval, activation, upgrade and retirement work.
- Providers: adapter registration, health check, capability mapping, security and evaluation work.
- Models: registry, benchmarks, pricing, health and router selection are auditable.

- Security: least privilege, secrets isolation and environment separation pass.
- Workflows: durable state, dependencies, retry, escalation and rollback pass.
- Delivery: code review, QA, security scan, staging and production gates pass.
- Operations: observability, cost controls, backups and disaster recovery are demonstrable.
- Extensibility: adding a provider, model, agent or product type does not require core architecture redesign.
- **v1.3 — see clause 53 for the fully measurable version of this matrix (pass/fail criteria with numbers, not just checkboxes).**

40. Final Enterprise Principle

- AI DIGITAL OFFICE OS is not tied to a single model, provider, agent, tool or software project. It is a reusable AI operating system and software factory capable of dynamically selecting AI capabilities, safely creating specialist agents, integrating future providers, orchestrating complete software projects and maintaining human-controlled governance throughout the lifecycle.

NEW IN v1.3 — Clauses 41 through 58 (Gap Closure)

41. Identity & Multi-Tenancy Model

- Tenancy pattern chosen: **shared database, shared schema, tenant_id column on every tenant-scoped table** (the standard scalable pattern for tens of thousands of tenants, versus one-database-per-tenant or one-schema-per-tenant which do not scale past low hundreds).
- Every tenant-scoped table's primary/foreign keys are paired with `tenant_id` in composite foreign keys or enforced via row-level security (RLS) policies, so referential integrity alone cannot create a cross-tenant reference — this closes the single most common multi-tenant RBAC bug (a syntactically valid but cross-tenant foreign key pairing).
- `organizations` (tenant root) → `users` (membership scoped to tenant, a user may belong to more than one tenant via a membership row) → `roles` (system template roles plus tenant-defined roles) → `permissions` (system-wide, tenant-agnostic definitions of “what the software can do”) → `role_permissions` and `user_roles` (both tenant-scoped junctions).
- `NULL tenant_id` is reserved exclusively for system-level template rows (agent templates, prompt templates, default roles) that tenants clone into their own scope; no operational data may carry a `NULL tenant_id`.

42. Agent-to-Agent Message Bus (Data Model Closure)

- `agent_messages` table implements clause 19's message contract as durable, queryable rows rather than in-memory conversational state: `task_id`, `workflow_id`, `sender_agent_id`, `receiver_agent_id`, `message_type`, `purpose`, `priority`, `security_level`, `input_payload`, `expected_output_schema`, `dependencies`, `deadline`, `artifact_reference`, `status`, `result`, `error`, `tenant_id`, `created_at`, `delivered_at`, `acknowledged_at`.
- Message delivery is at-least-once with idempotency keys; receivers must be idempotent on `message_id`.
- Optional external interoperability: an `a2a_capability_cards` table publishes this Office's agents as A2A-discoverable capability cards for approved partner-system integration, disabled by default (policy-gated).

43. Durable Workflow Engine (Data Model Closure)

- `workflow_registry` stores `workflow_id`, `project_id`, `tenant_id`, `workflow_type`, `definition_version`, `current_state`, `status` (RUNNING/PAUSED/COMPLETED/FAILED/CANCELLED/ESCALATED), `started_at`, `updated_at`, `completed_at`.
- `workflow_history` stores an append-only, replayable event log per workflow (`event_id`, `workflow_id`, `sequence_no`, `event_type`, `payload`, `created_at`) — this is what makes workflow state survive a process restart: on recovery, the engine replays `workflow_history` to reconstruct `current_state` rather than trusting only the mutable row.
- Each workflow step maps to one or more `task_registry` rows; each task execution attempt is a retryable “activity” with its own bounded retry policy (see clause 49), consistent with current durable-execution engine practice (e.g., Temporal-style activity/retry semantics) — the Office does not need to adopt a specific third-party engine, but the schema is compatible with one if adopted later.

44. RBAC — Full Data Model Closure

- Tables: `organizations`, `users`, `user_organization_membership`, `roles`, `permissions`, `role_permissions`, `user_roles` (all tenant-scoped except the system-wide `permissions` catalog).
- Policy decisions fixed explicitly (per current best practice, these must be decided up front, not left to ad-hoc convention): role ownership = system template roles + tenant-defined extensions; permission catalog is system-wide and version-controlled; a role is unusable until at least one permission is attached; every `user_roles` row carries `tenant_id` even though it is derivable, specifically so the foreign keys are composite and a cross-tenant row is rejected by the database itself, not just by application code.
- Enforcement happens at two layers: coarse authentication + tenant-membership check at the API gateway, fine-grained permission check at the service layer before any tool/model/data access — matching clause 13's chain (Agent → Role →

Permission → Tool → Resource → Action), extended to also cover human users of the control plane, not only agents.

45. Agent Activation Authority (Closes the Factory/Activation Ambiguity)

- Agent Factory pipeline output stops at `APPROVED`. A separate, explicit `activation_requests` record (subset of `approval_requests`, action type `AGENT_ACTIVATE`) is required to transition `APPROVED` → `ACTIVE`. This guarantees no pipeline — however automated — can put an agent into production without passing through the Approval Engine defined in clause 14.

46. Secrets Management — Full Data Model Closure

- The Office **never stores raw secret values** in its own database. `secrets_vault_references` stores only: `reference_id`, `tenant_id`, `secret_name`, `vault_path` (pointer into an external secrets manager such as a KMS/Vault product), `scope` (which agent_id/provider_id/tool it is bound to), `rotation_policy`, `last_rotated_at`, `expires_at`, `created_by`, `created_at`.
- Every dereference of a `vault_path` by an agent is written to `audit_events` with actor, purpose, and resource — satisfying clause 16's "every secret access is auditable" without ever placing a raw credential in an application table.
- Default credential lifetime: 15 minutes for AI provider keys issued to a running agent task, 24 hours for tool-gateway service credentials, both revocable immediately on policy violation.

47. Knowledge & Memory — Full Data Model Closure

- Three tiers, matching current agent-memory architecture practice:
 1. **Working memory** (`working_memory_cache`) — short-lived, per-task-run scratch state, TTL-bound (default 4 hours), never used for durable facts.
 2. **Durable structured memory** (`memory_facts`) — long-lived facts, preferences, project/organizational memory: `memory_id`, `tenant_id`, `scope` (AGENT/PROJECT/ORG), `subject_type`, `subject_id`, `fact`, `source_reference`, `confidence`, `created_at`, `expires_at` (nullable).
 3. **Semantic memory** (`memory_embeddings`) — pgvector-backed: `embedding_id`, `tenant_id`, `memory_fact_id` (nullable, may embed raw content instead), `content`, `embedding VECTOR(1536)`, `embedding_model`, `created_at`, with an HNSW index (`vector_cosine_ops`) for approximate nearest-neighbor retrieval at production scale, and a mandatory `embedding_model` column so mixed-model embeddings are never silently compared (a documented and common production failure mode).
- Retrieval flow (clause 22) routes by access pattern: hot per-turn context reads never touch pgvector; only semantic recall queries do — this avoids adding embedding latency to lookups a key-value or relational read can answer directly.

48. Artifact Registry — Full Data Model Closure

- `artifact_registry`: `artifact_id`, `tenant_id`, `project_id`, `task_id`, `agent_run_id`, `model_run_id`, `artifact_type` (CODE/DOC/DESIGN/CONFIG/REPORT/DATASET), `storage_uri`, `content_hash` (immutability proof), `version`, `status` (DRAFT/IN_REVIEW/APPROVED/RELEASED/DEPRECATED), `reviewed_by`, `approved_by`, `git_commit_ref`, `deployment_ref`, `created_at`.
- Immutability rule: once `status = APPROVED`, the row's `storage_uri` and `content_hash` become append-only — any further change creates a new `artifact_id` with `parent_artifact_id` set, preserving the full lineage chain required by clause 23.

49. Error Handling — Concrete Retry & Backoff Numbers

- Default retry policy for retryable error classes (TOOL_ERROR, MODEL_ERROR, PROVIDER_ERROR, NETWORK_ERROR, TIMEOUT, DATABASE_ERROR): bounded exponential backoff, base delay 500 ms, multiplier 2x, jitter ±20%, maximum 5 attempts, maximum total wait 60 seconds before moving to dead-letter / escalation.
- Non-retryable by default: VALIDATION_ERROR, AUTHORIZATION_ERROR, POLICY_ERROR, ARTIFACT_ERROR (immutability violations), DEPLOYMENT_ERROR without an approved rollback path — these escalate immediately rather than retrying.
- Idempotency keys are mandatory on every state-mutating task and message so bounded retries never double-execute a side effect (e.g., a duplicate deployment or duplicate external API call).

50. Disaster Recovery — Concrete RPO/RTO Targets

- Database: continuous WAL archiving, point-in-time recovery; **RPO ≤ 5 minutes, RTO ≤ 60 minutes** for the control-plane database.
- Artifacts (object storage): versioned storage with cross-region replication; **RPO ≤ 15 minutes, RTO ≤ 4 hours**.
- Full restore drill required at minimum **quarterly**, with results recorded as a `decision_records` entry (clause 24) including drill date, data restored, time taken, and pass/fail.

51. Configuration & Feature Flags — Full Data Model Closure

- `feature_flags`: `flag_id`, `tenant_id` (nullable = global default), `flag_key`, `flag_type` (BOOLEAN/PERCENTAGE/VARIANT), `default_value`, `tenant_override_value`, `environment`, `status`, `created_by`, `updated_at`.
- `configuration_versions`: `config_id`, `tenant_id`, `environment`, `version`, `payload`, `validated` (boolean), `rollback_of` (nullable)

self-reference), created_by, created_at — every configuration change is a new immutable version, never an in-place update, so rollback is always “activate a previous version,” never a destructive edit.

52. API Versioning & Security (Control-Plane Contract Hardening)

- All control-plane endpoints are versioned under `/v1`. Breaking changes require `/v2`; `/v1` is supported for a minimum 12-month deprecation window after `/v2` ships.
- Every endpoint requires bearer JWT authentication (OAuth2 client-credentials for service/agent callers, OAuth2 authorization-code for human users), scoped to a single `tenant_id` claim validated on every request — see the expanded `control_plane_openapi_v1.3.yaml` for the full `securitySchemes` and per-endpoint `requestBody` / `response` schemas, which were entirely absent from v1.2.
- Standard error envelope on every non-2xx response: `{ "error_code", "message", "correlation_id", "retryable" }`, matching the error classes in clause 31/49.

53. Acceptance Test Matrix — Measurable Version

Each v1.2 checkbox now carries a numeric or binary pass condition instead of an unqualified checkbox: - Agent creation → sandbox → evaluation → approval → activation completes end-to-end in a test tenant in **< 10 minutes** wall-clock for a standard-complexity agent spec. - Model router fallback triggers and completes within **< 3 seconds** of a primary provider health check failing. - A cross-tenant data-access attempt (row belonging to tenant A requested by a tenant B session) is rejected **100% of the time** in an automated test suite of at least 50 adversarial cases. - Workflow state survives a forced process restart mid-workflow and resumes from `workflow_history` with **zero task duplication**, verified by idempotency-key audit. - Disaster recovery drill meets the RPO/RTO numbers in clause 50 on at least the two most recent quarterly drills. - Full audit trail (`audit_events`) reconstructable for any artifact from creation to production release with **no gaps** in `correlation_id` chain, sampled monthly.

54. Interoperability Standards

- Internal tool discovery/invoke: Model Context Protocol (MCP) — consistent with clause 2/Layer 7’s existing MCP reference, now made the explicit standard rather than one option among several.
- External agent-to-agent interoperability (only when a specific partner integration is approved): A2A-style HTTP + JSON-RPC capability-card negotiation, disabled by default (clause 42).
- Rationale: standardizing on MCP for tools and an A2A-style contract for external agents avoids building a proprietary protocol that becomes a long-term maintenance and integration liability as the ecosystem consolidates around these patterns.

55. Observability — Agent-Specific Trace Requirements

- Beyond clause 29’s logs/metrics/traces/events/alerts: every model routing decision, every policy ALLOW/DENY/REQUIRE_APPROVAL decision, and every tool invocation must emit a structured trace event including the *alternatives considered and rejected* where applicable (e.g., which fallback models were skipped and why) — plain latency/error metrics are insufficient to debug non-deterministic agent behavior in production, a documented gap in standard APM tooling for agentic systems.

56. Indexing & Performance Baseline

- Every foreign key column is indexed by default. Every `tenant_id` column participates in a composite index with the table’s primary lookup key (e.g., (`tenant_id`, `status`) on `task_registry`) since virtually every production query is tenant-scoped.
- `memory_embeddings.embedding` uses an HNSW index (`m = 16`, `ef_construction = 64`) as a documented, tunable starting point) rather than IVFFlat, prioritizing recall/latency balance for an interactive agent workload over faster index build time.

57. Compliance & Audit Retention

- `audit_events` retention: minimum 400 days online, then cold-archive for a minimum of 5 years, configurable per tenant policy for regulated environments.
- Every RED-tier action (clause 14) is retained without any automatic expiry.

58. Explicit Non-Goals (Honesty Clause)

- This blueprint still does not include: actual infrastructure-as-code, actual cloud account/network topology, actual provider contracts/pricing agreements, actual penetration-test results, or actual production credentials. These remain environment-specific choices the package’s own README already flagged as out of scope in v1.2 — v1.3 does not manufacture fictitious values for these to appear more “complete”; it only closes the *architectural and data-model* gaps that were within a specification document’s proper scope.
-

NEW IN v1.4 — Clauses 59 through 76 (Integrity Closure)

59. Project Registry (Closes CRITICAL finding — was missing entirely)

- `project_registry` is now the authoritative record every `project_id` referenced across the blueprint, schema, and API must resolve to. Fields: `project_id`, `tenant_id`, `project_name`, `project_type`, `constitution_version`, `lifecycle_state` (matching clause 33's universal lifecycle), `risk_level`, `owner_user_id`, `repository_ref`, `environment_policy`, `created_at`, `updated_at`.
- `task_registry.project_id`, `workflow_registry.project_id`, `artifact_registry.project_id`, `decision_records.project_id`, and the new `deployment_registry / release_registry` (clause 66) all foreign-key into `project_registry` — closing the referential-integrity gap the review identified.

60. Agent Versioning — Immutable, Activatable by Version (Closes CRITICAL finding)

- `agent_registry` remains the identity record, but is no longer where “what is currently running” lives. `agent_versions` stores every version as an immutable row: `agent_version_id`, `agent_id`, `version`, `specification_hash`, `prompt_version`, `model_policy`, `permissions_snapshot`, `evaluation_score`, `lifecycle_state`, `created_at`.
- Updating an ACTIVE agent creates a **new** `agent_versions` row; it never overwrites an existing one (closing the exact gap the review flagged — v1.3's `agent_registry` alone could not express this).
- Activation (clause 45) now targets a specific `agent_version_id`, not just an `agent_id` — an agent can have version 3 active in production while version 4 is still in SANDBOX/EVALUATED.

61. Agent Run & Model Run Lineage (Closes CRITICAL finding)

- `agent_runs` and `model_runs` are added as the missing entities `artifact_registry.agent_run_id` and `model_run_id` already referenced by name in v1.3 but which did not exist as tables — this is precisely the “lineage references undefined entities” defect the review identified.
- `agent_runs` : `agent_run_id`, `tenant_id`, `task_id`, `agent_id`, `agent_version_id`, `status`, `started_at`, `completed_at`, `input_hash`, `output_hash`, `error`.
- `model_runs` : `model_run_id`, `tenant_id`, `agent_run_id`, `provider_id`, `model_id`, `routing_reason`, `input_tokens`, `output_tokens`, `estimated_cost`, `currency`, `started_at`, `completed_at`, `status`.
- With these two tables in place, the full lineage chain from clause 23 (Owner Command → Project → Task → Agent Run → Model Run → Tool Calls → Artifact → Review → Approval → Git Commit → Build → Deployment → Release) is now **actually resolvable by foreign-key traversal**, not just described in prose.

62. Full Tenant Row-Level Security (Closes CRITICAL finding)

- The v1.3 baseline of RLS on 4 tables is explicitly insufficient and is superseded here. **Every tenant-scoped operational table** must have RLS enabled with both a `USING` policy (governs which rows are visible) and a `WITH CHECK` policy (governs which rows can be inserted/updated) — `USING` alone permits a service-layer bug to write a cross-tenant row even if it can never read it back, which is still a serious integrity defect.
- This extends RLS to: prompts, policies, approvals, workflows, `workflow_history`, tasks, `agent_messages`, memory tables, secrets references, feature flags, and every RBAC junction table (`role_permissions` , `user_roles`) — not only the four tables named in v1.3.
- System-template rows (`NULL tenant_id` , used for agent/role templates) are explicitly constrained: they may only exist in tables designated as template-eligible (`agent_registry` , `roles`), and RLS policies elsewhere must reject a `NULL tenant_id` on any operational (non-template) table.

63. Agent Message Idempotency (Closes HIGH finding)

- `agent_messages` gains `idempotency_key` with a `UNIQUE (tenant_id, idempotency_key)` constraint — closing the gap where clause 42 promised idempotent messaging but the v1.3 table had no mechanism to enforce it.
- Duplicate delivery of the same (`tenant_id` , `idempotency_key`) pair returns the original message's recorded result rather than reprocessing.

64. Model Evaluation History (Closes HIGH finding)

- `model_evaluation_runs` stores every evaluation execution as an immutable row (`evaluation_id` , `provider_id` , `model_id` , `model_version` , `benchmark_suite` , `evaluator_version` , `score` , `results` , `executed_at`) instead of `model_registry` holding only a single current `evaluation_score` that silently loses history on re-evaluation.
- `model_registry.evaluation_score` becomes a denormalized “latest score” convenience field; `model_evaluation_runs` is the source of truth for clause 11's “versioned, rerunnable” promise.

65. Cost & Usage Ledger (Closes HIGH finding)

- `usage_events` is the authoritative, append-only ledger for every billable unit of work: `tenant_id`, `project_id`, `task_id`, `agent_id`, `agent_run_id`, `model_run_id`, `provider_id`, `model_id`, `input_tokens`, `output_tokens`, `request_count`, `actual_cost`,

currency, event_time.

- Cost Governance (clause 30) budget checks (WARNING/SOFT_LIMIT/HARD_LIMIT) now reconcile against `usage_events` rather than an unstated mechanism — closing the “cost governance defined but no ledger” gap.

66. Deployment & Release Registry (Closes HIGH finding)

- `deployment_registry` (`deployment_id`, `tenant_id`, `project_id`, `release_id`, `environment`, `strategy`, `status`, `artifact_refs`, `approval_request_id`, `rollback_target`, `started_at`, `completed_at`) and `release_registry` (`release_id`, `tenant_id`, `project_id`, `version`, `artifact_refs`, `status`, `approved_by`, `created_at`) replace the free-text `deployment_ref` string on `artifact_registry` that v1.3 left unstructured.
- Every deployment now carries an explicit, queryable rollback target and links back to the approval that authorized it (clause 14/27).

67. MCP Tool Registry & Agent-Tool Bindings (Closes HIGH finding)

- `mcp_server_registry` (`mcp_server_id`, `tenant_id`, `server_name`, `endpoint`, `version`, `trust_level`, `enabled`) and `tool_registry` (`tool_id`, `mcp_server_id`, `tool_name`, `version`, `input_schema`, `output_schema`, `risk_level`, `enabled`) give MCP — already named as the explicit interoperability standard in clause 54 — a persistent, versioned registry instead of only a narrative reference.
- `agent_tool_bindings` (`tenant_id`, `agent_id`, `tool_id`, `allowed_actions`) replaces the loose `allowed_tools` JSONB array on `agent_registry` with a tenant-scoped, policy-checkable binding table, so RBAC and Policy Engine decisions can bind to a stable `tool_id` rather than a free-text tool name.

68. Complete API Lifecycle Coverage (Closes HIGH finding)

- The v1.3 contract (18 endpoints) is extended with the full resource lifecycle the review found missing: project CRUD, workflow create/cancel/retry/escalate, agent version create/list/activate, agent message send, artifact create + lineage retrieval, provider/model listing, model evaluation trigger + history, tool/MCP server listing, usage/cost queries, deployment create/get/rollback, and policy/routing decision queries. See `control_plane_openapi_v1.4.yaml` for the full, expanded contract (34 endpoints).
- Every state-changing endpoint now requires an `Idempotency-Key` header, consistent with clause 63 and the existing `task_registry.idempotency_key` pattern from v1.3.

69. Schema / API / Domain-Model Consistency Gate (Closes HIGH finding)

- The review found `AgentRecord` and other OpenAPI schemas narrower than their SQL counterparts — a drift risk that compounds silently over time. v1.4 mandates: API schemas are either generated from a single canonical domain model, or a CI-enforced schema-diff test fails the build whenever a SQL column and its corresponding OpenAPI field diverge.
- This is a process/governance requirement, not a table — it belongs in the implementation phase’s CI configuration, but is recorded here as a binding acceptance criterion (see clause 76).

70. Authentication Contract Detail (Closes MEDIUM finding)

- JWT bearer tokens must specify: `iss` (issuer, the Office’s own identity provider or an approved external IdP), `aud` (audience, scoped to this control plane), `exp / iat` with a maximum token lifetime of 15 minutes for service/agent callers and 8 hours for human user sessions, and a `scope` claim listing granted permission scopes.
- JWKS (JSON Web Key Set) rotation: keys rotate on a maximum 90-day cycle with a minimum 7-day overlap window where both old and new keys validate, so in-flight tokens are never invalidated mid-rotation.
- Clock-skew tolerance: ± 60 seconds on `exp / iat` validation.
- Service/agent callers and human users are distinguished by a required `actor_type` claim (`SERVICE` | `HUMAN`) — Policy Engine decisions (clause 12) may condition on this claim directly.

71. Session Management Boundary (Closes MEDIUM finding)

- Decision made explicitly rather than left implicit (per the review’s two offered options): session and refresh-token management is **delegated to the external identity provider** referenced in clause 70’s `iss` claim. AI DIGITAL OFFICE OS does not implement its own `session_registry` table or session-revocation store in this specification.
- The boundary this creates: token issuance, refresh, and revocation are the IdP’s responsibility; the Office’s own responsibility is limited to validating a presented token against the current JWKS and honoring `REQUIRE_REAUTH` -style signals the IdP may emit. If a future version chooses to bring session management in-house instead, that is an explicit architectural change requiring its own review — not a silent addition.

72. Policy & Routing Decision Records (Closes MEDIUM finding)

- `policy_decision_records` (`policy_decision_id`, `tenant_id`, `task_id`, `agent_id`, `tool_id`, `model_id`, `provider_id`, `decision`, `policy_version`, `alternatives`, `rejection_reasons`, `created_at`) and `routing_decision_records` (`routing_decision_id`, `tenant_id`, `task_id`, `agent_id`, `selected_provider`, `selected_model`, `candidate_models`, `reason`, `policy_result`, `created_at`) promote

clause 12 and clause 10's decisions from "auditable via `audit_events.payload`" to first-class, structurally queryable records — needed for deterministic incident reconstruction, per the review's specific concern.

73. Evidence Discipline — TARGET / TEST REQUIRED / VERIFIED (Closes MEDIUM finding)

- Every acceptance-checklist item (clause 39/53, and `implementation_acceptance_checklist_v1.4.md`) must now carry one of exactly three evidence labels:
 - **TARGET** — a design goal or NFR baseline (e.g., the numbers in `nfr_slo_v1.4.md`) not yet tested against a running system.
 - **TEST REQUIRED** — a specific automated or manual test exists or is planned, but has not yet been executed to produce a result.
 - **VERIFIED** — an executed test has produced recorded evidence (a test run ID, timestamp, and result) confirming the item.
- No item may be marked VERIFIED on the strength of blueprint text alone. This directly closes the review's finding that some v1.3 checklist items "read like an achieved production state" when they were still design targets.

74. Specification vs. Implementation Package Boundary (Closes MEDIUM finding)

- Confirmed and made explicit: this document, its schema, and its API contract remain a **specification and data-model package**. They intentionally exclude infrastructure-as-code, cloud account/network topology, real provider contracts and pricing, penetration-test results, and production credentials.
- When implementation begins, a **separate, distinct implementation-phase package** should be created to hold those environment-specific artifacts (IaC, deployment topology, secrets integration, real provider adapter code, and executed test evidence). That package is explicitly out of scope for v1.4 and is not fabricated here.

75. Product Independence — Decoupling from Any Named Downstream Project (User-Directed Change)

- This clause formalizes, at the "new clauses" level, the same instruction already applied directly to clause 34: AI DIGITAL OFFICE OS carries **no architectural coupling to any specific downstream product**. The Agent Factory, Model Router, Tool Gateway, Policy Engine, QA, Security, Artifact, and Governance systems are all product-agnostic by construction (clause 37's consistency rules already required this; v1.4 removes the one place — clause 34 — where a specific product had been named as a privileged example).
- Practical effect: `project_registry` (clause 59) treats every project type identically. There is no schema column, no API branch, and no agent template that is specific to any one named product. A tenant may build any product, including none at all, without that decision touching the core specification.

76. Implementation-Lock Gate

Per the independent v1.4 review's own stated gate, this blueprint may be locked as the implementation baseline only once **all** of the following hold, each backed by VERIFIED evidence per clause 73 (not merely present in the schema): 1. The SQL schema, OpenAPI contract, and canonical domain model pass an automated consistency check with zero drift (clause 69). 2. Every tenant-scoped table has RLS enabled with both `USING` and `WITH CHECK` policies, and a ≥50-case adversarial cross-tenant test suite passes at 100% (clause 62). 3. Artifact lineage resolves end-to-end via foreign-key traversal alone, with no unresolved reference (clause 61). 4. Agent versions are immutable rows, and production activation always targets a specific `agent_version_id` (clause 60). 5. The acceptance checklist has every item labeled TARGET, TEST REQUIRED, or VERIFIED, with no unlabeled or ambiguously-labeled items (clause 73). Until all five hold with recorded evidence, this specification is **implementation-lock candidate**, not implementation-locked — consistent with the independent review's final verdict.

Section C — Database Schema v1.4 (Full SQL — 41 Tables, Full RLS)

```
-- =====
-- AI DIGITAL OFFICE OS v1.3 – Full Logical Schema
-- Closes all gaps identified in the v1.2 review: multi-tenancy, RBAC,
-- workflow durability, agent messaging, tiered memory, artifact lineage,
-- secrets references, feature flags. Every business table carries
-- tenant_id; every junction table carries tenant_id in its composite
-- foreign keys so referential integrity alone cannot create a
-- cross-tenant privilege escalation.
-- Target: PostgreSQL 15+ with the pgvector extension.
-- =====

CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS vector;

-- =====
-- 1. IDENTITY & MULTI-TENANCY (Blueprint clause 41)
-- =====

CREATE TABLE organizations (
```

```

tenant_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
org_name VARCHAR(160) NOT NULL,
org_slug VARCHAR(80) NOT NULL UNIQUE,
status VARCHAR(32) NOT NULL DEFAULT 'ACTIVE',          -- ACTIVE | SUSPENDED | ARCHIVED
plan_tier VARCHAR(32) DEFAULT 'STANDARD',
data_residency VARCHAR(32),
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE users (
  user_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(256) NOT NULL UNIQUE,
  display_name VARCHAR(160),
  auth_provider VARCHAR(64) NOT NULL DEFAULT 'PASSWORD', -- PASSWORD | SSO_OIDC | SSO_SAML
  status VARCHAR(32) NOT NULL DEFAULT 'ACTIVE',
  mfa_enabled BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- A user may belong to more than one tenant; each membership row is the
-- tenant-scoping anchor for every role assignment below.
CREATE TABLE user_organization_membership (
  membership_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  user_id UUID NOT NULL REFERENCES users(user_id),
  status VARCHAR(32) NOT NULL DEFAULT 'ACTIVE',          -- ACTIVE | INVITED | SUSPENDED | REMOVED
  invited_by UUID REFERENCES users(user_id),
  joined_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE (tenant_id, user_id)
);

-- Permissions are system-wide: they describe what the software CAN do,
-- independent of any tenant. Tenants compose them into roles.
CREATE TABLE permissions (
  permission_id VARCHAR(96) PRIMARY KEY,                -- e.g. 'agent:create', 'deployment:approve'
  resource VARCHAR(64) NOT NULL,
  action VARCHAR(32) NOT NULL,                          --
  READ|WRITE|CREATE|UPDATE|DELETE|EXECUTE|DEPLOY|APPROVE|EXPORT|ADMINISTER
  description TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Roles are tenant-scoped (NULL tenant_id = system template a tenant clones).
CREATE TABLE roles (
  role_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID REFERENCES organizations(tenant_id),    -- NULL = system template
  role_name VARCHAR(96) NOT NULL,
  is_default BOOLEAN NOT NULL DEFAULT FALSE,            -- default role granted on membership join
  description TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE (tenant_id, role_name),
  UNIQUE (tenant_id, role_id)
);

-- Composite FK on (tenant_id, role_id) prevents a role from tenant B
-- being attached under tenant A's scope.
CREATE TABLE role_permissions (
  tenant_id UUID,                                       -- mirrors roles.tenant_id (NULL for system roles)
  role_id UUID NOT NULL,
  permission_id VARCHAR(96) NOT NULL REFERENCES permissions(permission_id),
  granted_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (role_id, permission_id),
  FOREIGN KEY (tenant_id, role_id) REFERENCES roles(tenant_id, role_id)
);

CREATE TABLE user_roles (
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  user_id UUID NOT NULL REFERENCES users(user_id),
  role_id UUID NOT NULL,
  assigned_by UUID REFERENCES users(user_id),
  assigned_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (tenant_id, user_id, role_id),
  FOREIGN KEY (tenant_id, role_id) REFERENCES roles(tenant_id, role_id)
);

-- =====
-- 2. PROVIDER / MODEL / AGENT REGISTRIES (Blueprint clauses 6-11, 41)
-- =====

CREATE TABLE provider_registry (
  provider_id VARCHAR(64) PRIMARY KEY,
  provider_name VARCHAR(128) NOT NULL,
  provider_type VARCHAR(64) NOT NULL,
  adapter_type VARCHAR(128) NOT NULL,
  protocol VARCHAR(64),
  base_endpoint TEXT,
  authentication_method VARCHAR(64),
  supported_capabilities JSONB NOT NULL DEFAULT '[]',
  supported_models JSONB NOT NULL DEFAULT '[]',
  rate_limits JSONB,
  quota_rules JSONB,

```

```

pricing_rules JSONB,
privacy_classification VARCHAR(32),
data_residency JSONB,
availability VARCHAR(32) DEFAULT 'ACTIVE',
health_status VARCHAR(32) DEFAULT 'UNKNOWN',
version VARCHAR(32),
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE model_registry (
model_id VARCHAR(64) PRIMARY KEY,
provider_id VARCHAR(64) NOT NULL REFERENCES provider_registry(provider_id),
model_name VARCHAR(160) NOT NULL,
model_version VARCHAR(64),
capabilities JSONB NOT NULL DEFAULT '[]',
context_window INT,
input_types JSONB,
output_types JSONB,
tool_calling BOOLEAN DEFAULT FALSE,
structured_output BOOLEAN DEFAULT FALSE,
vision BOOLEAN DEFAULT FALSE,
coding BOOLEAN DEFAULT FALSE,
reasoning BOOLEAN DEFAULT FALSE,
research BOOLEAN DEFAULT FALSE,
latency_profile JSONB,
cost_profile JSONB,
privacy_classification VARCHAR(32),
local_cloud VARCHAR(16),
availability VARCHAR(32) DEFAULT 'ACTIVE',
health_status VARCHAR(32) DEFAULT 'UNKNOWN',
evaluation_score NUMERIC(6,3),
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);

-- NULL tenant_id = system agent template a tenant may clone; every
-- executable/running agent row MUST have a non-null tenant_id.
CREATE TABLE agent_registry (
agent_id VARCHAR(64) PRIMARY KEY,
tenant_id UUID REFERENCES organizations(tenant_id),
agent_name VARCHAR(128) NOT NULL,
department VARCHAR(64) NOT NULL,
role VARCHAR(128) NOT NULL,
purpose TEXT,
capabilities JSONB NOT NULL DEFAULT '[]',
allowed_tools JSONB NOT NULL DEFAULT '[]',
permissions JSONB NOT NULL DEFAULT '[]',
data_access JSONB NOT NULL DEFAULT '[]',
preferred_capabilities JSONB NOT NULL DEFAULT '[]',
preferred_provider VARCHAR(64) REFERENCES provider_registry(provider_id),
preferred_model VARCHAR(64) REFERENCES model_registry(model_id),
fallback_models JSONB NOT NULL DEFAULT '[]',
input_schema JSONB,
output_schema JSONB,
security_level VARCHAR(16) DEFAULT 'GREEN',
lifecycle_state VARCHAR(32) DEFAULT 'DRAFT',
--
DRAFT|SANDBOX|TESTED|EVALUATED|APPROVED|ACTIVE|UPDATED|DEPRECATED|RETIRED
status VARCHAR(32) DEFAULT 'INACTIVE',
version VARCHAR(32) DEFAULT '1.0.0',
parent_agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
evaluation_score NUMERIC(6,3),
success_rate NUMERIC(6,3),
average_latency_ms INT,
average_cost NUMERIC(12,6),
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE prompt_registry (
prompt_id VARCHAR(64) PRIMARY KEY,
tenant_id UUID REFERENCES organizations(tenant_id),
agent_id VARCHAR(64) NOT NULL REFERENCES agent_registry(agent_id),
version VARCHAR(32) NOT NULL,
system_instruction TEXT NOT NULL,
variables JSONB,
input_contract JSONB,
output_contract JSONB,
evaluation_score NUMERIC(6,3),
security_classification VARCHAR(32),
changelog TEXT,
rollback_version VARCHAR(32),
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- 3. POLICY, APPROVAL & ACTIVATION AUTHORITY (clauses 12-14, 45)
-- =====

CREATE TABLE policy_registry (
policy_id VARCHAR(64) PRIMARY KEY,
tenant_id UUID REFERENCES organizations(tenant_id),

```

```

policy_name VARCHAR(160) NOT NULL,
policy_version VARCHAR(32) NOT NULL,
rules JSONB NOT NULL,
status VARCHAR(32) DEFAULT 'ACTIVE',
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE approval_requests (
request_id VARCHAR(64) PRIMARY KEY,
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
task_id VARCHAR(64),
requester VARCHAR(128),
agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
action VARCHAR(128) NOT NULL,
risk_level VARCHAR(16) NOT NULL,
reason TEXT,
approver VARCHAR(128),
decision VARCHAR(32),
expires_at TIMESTAMPTZ,
execution_result JSONB,
created_at TIMESTAMPTZ DEFAULT now(),
decided_at TIMESTAMPTZ
);

-- =====
-- 4. TASK ENGINE & DURABLE WORKFLOWS (clauses 17-18, 43)
-- =====

CREATE TABLE workflow_registry (
workflow_id VARCHAR(64) PRIMARY KEY,
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
project_id VARCHAR(64),
workflow_type VARCHAR(96) NOT NULL,
definition_version VARCHAR(32) NOT NULL,
current_state JSONB,
status VARCHAR(32) DEFAULT 'RUNNING',
started_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now(),
completed_at TIMESTAMPTZ
);

-- Append-only, replayable event log: on process restart, replay this to
-- reconstruct workflow_registry.current_state rather than trusting only
-- the mutable row (durable-execution pattern).

CREATE TABLE workflow_history (
event_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
workflow_id VARCHAR(64) NOT NULL REFERENCES workflow_registry(workflow_id),
sequence_no BIGINT NOT NULL,
event_type VARCHAR(64) NOT NULL,
payload JSONB,
created_at TIMESTAMPTZ DEFAULT now(),
UNIQUE (workflow_id, sequence_no)
);

CREATE TABLE task_registry (
task_id VARCHAR(64) PRIMARY KEY,
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
project_id VARCHAR(64),
workflow_id VARCHAR(64) REFERENCES workflow_registry(workflow_id),
parent_task_id VARCHAR(64) REFERENCES task_registry(task_id),
assigned_agent VARCHAR(64) REFERENCES agent_registry(agent_id),
required_capability VARCHAR(128),
priority VARCHAR(16),
risk_level VARCHAR(16),
security_level VARCHAR(16),
dependencies JSONB DEFAULT '[]',
input JSONB,
expected_output JSONB,
status VARCHAR(32) DEFAULT 'CREATED',
retry_count INT DEFAULT 0,
idempotency_key VARCHAR(128) NOT NULL,
deadline TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now(),
UNIQUE (tenant_id, idempotency_key)
);

-- =====
-- 5. AGENT-TO-AGENT MESSAGE BUS (clause 42)
-- =====

CREATE TABLE agent_messages (
message_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
task_id VARCHAR(64) REFERENCES task_registry(task_id),
workflow_id VARCHAR(64) REFERENCES workflow_registry(workflow_id),
sender_agent_id VARCHAR(64) NOT NULL REFERENCES agent_registry(agent_id),
receiver_agent_id VARCHAR(64) NOT NULL REFERENCES agent_registry(agent_id),
message_type VARCHAR(64) NOT NULL,

```

```

purpose TEXT,
priority VARCHAR(16),
security_level VARCHAR(16),
input_payload JSONB,
expected_output_schema JSONB,
dependencies JSONB DEFAULT '[]',
deadline TIMESTAMPTZ,
artifact_reference VARCHAR(64),
status VARCHAR(32) DEFAULT 'SENT', -- SENT|DELIVERED|ACKNOWLEDGED|FAILED
result JSONB,
error JSONB,
created_at TIMESTAMPTZ DEFAULT now(),
delivered_at TIMESTAMPTZ,
acknowledged_at TIMESTAMPTZ
);

-- Optional external interoperability surface (disabled by default; see
-- clause 42/54). Publishes this Office's agents as A2A-discoverable cards.
CREATE TABLE a2a_capability_cards (
card_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
agent_id VARCHAR(64) NOT NULL REFERENCES agent_registry(agent_id),
card_payload JSONB NOT NULL,
enabled BOOLEAN NOT NULL DEFAULT FALSE,
published_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- 6. KNOWLEDGE & MEMORY – TIERED (clause 47)
-- =====

-- Tier 1: working memory (fast, short-lived; TTL enforced by application
-- job or Postgres partition rotation, default TTL 4 hours).
CREATE TABLE working_memory_cache (
cache_key VARCHAR(256) PRIMARY KEY,
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
task_id VARCHAR(64),
payload JSONB NOT NULL,
expires_at TIMESTAMPTZ NOT NULL,
created_at TIMESTAMPTZ DEFAULT now()
);

-- Tier 2: durable structured memory.
CREATE TABLE memory_facts (
memory_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
scope VARCHAR(16) NOT NULL, -- AGENT|PROJECT|ORG
subject_type VARCHAR(32) NOT NULL,
subject_id VARCHAR(64) NOT NULL,
fact TEXT NOT NULL,
source_reference VARCHAR(128),
confidence NUMERIC(4,3),
created_at TIMESTAMPTZ DEFAULT now(),
expires_at TIMESTAMPTZ
);

-- Tier 3: semantic memory (pgvector, HNSW-indexed).
CREATE TABLE memory_embeddings (
embedding_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
memory_fact_id UUID REFERENCES memory_facts(memory_id),
content TEXT NOT NULL,
embedding VECTOR(1536) NOT NULL,
embedding_model VARCHAR(96) NOT NULL, -- prevents mixing incompatible embedding spaces
created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_memory_embeddings_hnsw
ON memory_embeddings USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- =====
-- 7. ARTIFACT REGISTRY (clause 48)
-- =====

CREATE TABLE artifact_registry (
artifact_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
project_id VARCHAR(64),
task_id VARCHAR(64) REFERENCES task_registry(task_id),
agent_run_id VARCHAR(64),
model_run_id VARCHAR(64),
artifact_type VARCHAR(32) NOT NULL, -- CODE|DOC|DESIGN|CONFIG|REPORT|DATASET
storage_uri TEXT NOT NULL,
content_hash VARCHAR(128) NOT NULL,
version VARCHAR(32) NOT NULL DEFAULT '1.0.0',
status VARCHAR(32) DEFAULT 'DRAFT', -- DRAFT|IN_REVIEW|APPROVED|RELEASED|DEPRECATED
parent_artifact_id UUID REFERENCES artifact_registry(artifact_id),
reviewed_by VARCHAR(128),
approved_by VARCHAR(128),
git_commit_ref VARCHAR(128),
deployment_ref VARCHAR(128),

```

```

        created_at TIMESTAMPTZ DEFAULT now()
    );

-- =====
-- 8. DECISION RECORDS & AUDIT (clauses 24, 29, 50, 57)
-- =====

CREATE TABLE decision_records (
    decision_id VARCHAR(64) PRIMARY KEY,
    tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
    project_id VARCHAR(64),
    decision TEXT NOT NULL,
    reason TEXT,
    alternatives JSONB,
    evidence JSONB,
    agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
    model_id VARCHAR(64) REFERENCES model_registry(model_id),
    author VARCHAR(128),
    approval JSONB,
    impact TEXT,
    status VARCHAR(32),
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE audit_events (
    event_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID REFERENCES organizations(tenant_id),
    correlation_id VARCHAR(128),
    event_type VARCHAR(128) NOT NULL,
    actor_type VARCHAR(32),
    actor_id VARCHAR(128),
    project_id VARCHAR(64),
    task_id VARCHAR(64),
    workflow_id VARCHAR(64),
    payload JSONB,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- 9. SECRETS REFERENCES (no raw secrets ever stored – clause 46)
-- =====

CREATE TABLE secrets_vault_references (
    reference_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
    secret_name VARCHAR(160) NOT NULL,
    vault_path TEXT NOT NULL, -- pointer only, e.g. into an external KMS/Vault
    scope_agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
    scope_provider_id VARCHAR(64) REFERENCES provider_registry(provider_id),
    scope_tool VARCHAR(96),
    rotation_policy VARCHAR(64),
    last_rotated_at TIMESTAMPTZ,
    expires_at TIMESTAMPTZ,
    created_by VARCHAR(128),
    created_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- 10. CONFIGURATION & FEATURE FLAGS (clause 51)
-- =====

CREATE TABLE feature_flags (
    flag_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID REFERENCES organizations(tenant_id), -- NULL = global default
    flag_key VARCHAR(128) NOT NULL,
    flag_type VARCHAR(16) NOT NULL DEFAULT 'BOOLEAN', -- BOOLEAN|PERCENTAGE|VARIANT
    default_value JSONB NOT NULL,
    tenant_override_value JSONB,
    environment VARCHAR(32) NOT NULL DEFAULT 'production',
    status VARCHAR(32) DEFAULT 'ACTIVE',
    created_by VARCHAR(128),
    updated_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE (tenant_id, flag_key, environment)
);

CREATE TABLE configuration_versions (
    config_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID REFERENCES organizations(tenant_id),
    environment VARCHAR(32) NOT NULL,
    version VARCHAR(32) NOT NULL,
    payload JSONB NOT NULL,
    validated BOOLEAN NOT NULL DEFAULT FALSE,
    rollback_of UUID REFERENCES configuration_versions(config_id),
    created_by VARCHAR(128),
    created_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- 11. PROJECT REGISTRY (v1.4 clause 59 – closes CRITICAL finding)
-- Authoritative record for every project_id referenced elsewhere.
-- =====

CREATE TABLE project_registry (

```

```

project_id VARCHAR(64) PRIMARY KEY,
tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
project_name VARCHAR(200) NOT NULL,
project_type VARCHAR(64) NOT NULL,           -- free-form; no product type is special-cased (clause 75)
constitution_version VARCHAR(32),
lifecycle_state VARCHAR(32) NOT NULL DEFAULT 'DISCOVERY',
risk_level VARCHAR(16) NOT NULL DEFAULT 'GREEN',
owner_user_id UUID REFERENCES users(user_id),
repository_ref VARCHAR(256),
environment_policy JSONB,
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);

-- Retrofit FK integrity onto existing project_id columns (v1.3 tables had
-- free-text project id with no referenced table to point to).
ALTER TABLE task_registry ADD CONSTRAINT fk_task_project FOREIGN KEY (project_id) REFERENCES project_registry(project_id)
ALTER TABLE workflow_registry ADD CONSTRAINT fk_workflow_project FOREIGN KEY (project_id) REFERENCES
project_registry(project_id);
ALTER TABLE artifact_registry ADD CONSTRAINT fk_artifact_project FOREIGN KEY (project_id) REFERENCES
project_registry(project_id);
ALTER TABLE decision_records ADD CONSTRAINT fk_decision_project FOREIGN KEY (project_id) REFERENCES
project_registry(project_id);

-- =====
-- 12. AGENT VERSIONING (v1.4 clause 60 – closes CRITICAL finding)
-- Updating an ACTIVE agent creates a new row here; agent_registry itself
-- is never overwritten in place.
-- =====

CREATE TABLE agent_versions (
  agent_version_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  agent_id VARCHAR(64) NOT NULL REFERENCES agent_registry(agent_id),
  version VARCHAR(32) NOT NULL,
  specification_hash VARCHAR(128) NOT NULL,
  prompt_version VARCHAR(32),
  model_policy JSONB,
  permissions_snapshot JSONB,
  evaluation_score NUMERIC(6,3),
  lifecycle_state VARCHAR(32) NOT NULL,       -- DRAFT|SANDBOX|TESTED|EVALUATED|APPROVED|ACTIVE|DEPRECATED|RETIRED
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE (tenant_id, agent_id, version)
);

-- Activation now targets a specific version, not just the agent identity.
ALTER TABLE agent_registry ADD COLUMN active_agent_version_id UUID REFERENCES agent_versions(agent_version_id);

-- =====
-- 13. AGENT RUN & MODEL RUN LINEAGE (v1.4 clause 61 – closes CRITICAL finding)
-- These are the entities artifact_registry.agent_run_id / model_run_id
-- already referenced by name in v1.3 without a backing table.
-- =====

CREATE TABLE agent_runs (
  agent_run_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  task_id VARCHAR(64) REFERENCES task_registry(task_id),
  agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
  agent_version_id UUID REFERENCES agent_versions(agent_version_id),
  status VARCHAR(32) NOT NULL,
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  input_hash VARCHAR(128),
  output_hash VARCHAR(128),
  error JSONB
);

CREATE TABLE model_runs (
  model_run_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  agent_run_id VARCHAR(64) REFERENCES agent_runs(agent_run_id),
  provider_id VARCHAR(64) REFERENCES provider_registry(provider_id),
  model_id VARCHAR(64) REFERENCES model_registry(model_id),
  routing_reason TEXT,
  input_tokens BIGINT,
  output_tokens BIGINT,
  estimated_cost NUMERIC(18,8),
  currency VARCHAR(8) DEFAULT 'USD',
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  status VARCHAR(32)
);

-- Retrofit real FKs onto artifact_registry's previously-unbacked columns.
-- artifact_registry.agent_run_id / model_run_id were VARCHAR(64) in v1.3
-- with no referenced table; they now point at the tables above.
ALTER TABLE artifact_registry ADD CONSTRAINT fk_artifact_agent_run FOREIGN KEY (agent_run_id) REFERENCES
agent_runs(agent_run_id);
ALTER TABLE artifact_registry ADD CONSTRAINT fk_artifact_model_run FOREIGN KEY (model_run_id) REFERENCES
model_runs(model_run_id);

-- =====

```

```

-- 14. MODEL EVALUATION HISTORY (v1.4 clause 64 – closes HIGH finding)
-- =====

CREATE TABLE model_evaluation_runs (
  evaluation_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  provider_id VARCHAR(64) REFERENCES provider_registry(provider_id),
  model_id VARCHAR(64) REFERENCES model_registry(model_id),
  model_version VARCHAR(64),
  benchmark_suite VARCHAR(128) NOT NULL,
  evaluator_version VARCHAR(64),
  score NUMERIC(8,4),
  results JSONB,
  executed_at TIMESTAMPTZ DEFAULT now()
);

-- Per-metric breakdown (coding/reasoning/tool-use/etc. – clause 11), so a
-- single evaluation_id can carry more than one named metric.
CREATE TABLE model_evaluation_metrics (
  metric_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  evaluation_id UUID NOT NULL REFERENCES model_evaluation_runs(evaluation_id),
  metric_name VARCHAR(96) NOT NULL,          -- e.g. 'coding', 'reasoning', 'tool_precision'
  metric_value NUMERIC(10,4) NOT NULL,
  unit VARCHAR(32)
);

-- =====
-- 15. COST & USAGE LEDGER (v1.4 clause 65 – closes HIGH finding)
-- =====

CREATE TABLE usage_events (
  usage_event_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  project_id VARCHAR(64) REFERENCES project_registry(project_id),
  task_id VARCHAR(64) REFERENCES task_registry(task_id),
  agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
  agent_run_id VARCHAR(64) REFERENCES agent_runs(agent_run_id),
  model_run_id VARCHAR(64) REFERENCES model_runs(model_run_id),
  provider_id VARCHAR(64) REFERENCES provider_registry(provider_id),
  model_id VARCHAR(64) REFERENCES model_registry(model_id),
  input_tokens BIGINT DEFAULT 0,
  output_tokens BIGINT DEFAULT 0,
  request_count INT DEFAULT 1,
  actual_cost NUMERIC(18,8),
  currency VARCHAR(8) DEFAULT 'USD',
  billing_status VARCHAR(32) DEFAULT 'UNBILLED',
  event_time TIMESTAMPTZ DEFAULT now()
);

-- =====
-- 16. DEPLOYMENT & RELEASE REGISTRY (v1.4 clause 66 – closes HIGH finding)
-- Replaces artifact_registry's free-text deployment_ref.
-- =====

CREATE TABLE release_registry (
  release_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  project_id VARCHAR(64) REFERENCES project_registry(project_id),
  version VARCHAR(64) NOT NULL,
  artifact_refs JSONB NOT NULL,
  status VARCHAR(32) NOT NULL,
  approved_by VARCHAR(128),
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE deployment_registry (
  deployment_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  project_id VARCHAR(64) REFERENCES project_registry(project_id),
  release_id VARCHAR(64) REFERENCES release_registry(release_id),
  environment VARCHAR(32) NOT NULL,
  strategy VARCHAR(32) NOT NULL,          -- standard|rolling|blue_green|canary (clause 27)
  status VARCHAR(32) NOT NULL,
  artifact_refs JSONB,
  approval_request_id VARCHAR(64) REFERENCES approval_requests(request_id),
  rollback_target VARCHAR(64) REFERENCES deployment_registry(deployment_id),
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ
);

-- =====
-- 17. MCP TOOL REGISTRY & AGENT-TOOL BINDINGS (v1.4 clause 67 – closes HIGH finding)
-- =====

CREATE TABLE mcp_server_registry (
  mcp_server_id VARCHAR(64) PRIMARY KEY,
  tenant_id UUID REFERENCES organizations(tenant_id),          -- NULL = platform-wide server
  server_name VARCHAR(160) NOT NULL,
  endpoint TEXT NOT NULL,
  version VARCHAR(64),
  trust_level VARCHAR(32) NOT NULL DEFAULT 'UNTRUSTED',
  enabled BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT now()
);

```

```

CREATE TABLE tool_registry (
  tool_id VARCHAR(64) PRIMARY KEY,
  mcp_server_id VARCHAR(64) REFERENCES mcp_server_registry(mcp_server_id),
  tool_name VARCHAR(160) NOT NULL,
  version VARCHAR(64),
  input_schema JSONB,
  output_schema JSONB,
  risk_level VARCHAR(16) NOT NULL DEFAULT 'GREEN',
  enabled BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Supersedes agent_registry.allowed_tools JSONB as the policy-checkable
-- binding surface; allowed_tools is retained for backward compatibility
-- but agent_tool_bindings is the authoritative source going forward.
CREATE TABLE agent_tool_bindings (
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  agent_id VARCHAR(64) NOT NULL REFERENCES agent_registry(agent_id),
  tool_id VARCHAR(64) NOT NULL REFERENCES tool_registry(tool_id),
  allowed_actions JSONB NOT NULL DEFAULT '[]',
  PRIMARY KEY (tenant_id, agent_id, tool_id)
);

-- =====
-- 18. POLICY & ROUTING DECISION RECORDS (v1.4 clause 72 – closes MEDIUM finding)
-- =====

CREATE TABLE policy_decision_records (
  policy_decision_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  task_id VARCHAR(64) REFERENCES task_registry(task_id),
  agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
  tool_id VARCHAR(64) REFERENCES tool_registry(tool_id),
  model_id VARCHAR(64) REFERENCES model_registry(model_id),
  provider_id VARCHAR(64) REFERENCES provider_registry(provider_id),
  decision VARCHAR(32) NOT NULL, -- ALLOW|DENY|REQUIRE_APPROVAL|REQUIRE_ESCALATION
  policy_version VARCHAR(32) NOT NULL,
  alternatives JSONB,
  rejection_reasons JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE routing_decision_records (
  routing_decision_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID NOT NULL REFERENCES organizations(tenant_id),
  task_id VARCHAR(64) REFERENCES task_registry(task_id),
  agent_id VARCHAR(64) REFERENCES agent_registry(agent_id),
  selected_provider VARCHAR(64) REFERENCES provider_registry(provider_id),
  selected_model VARCHAR(64) REFERENCES model_registry(model_id),
  candidate_models JSONB,
  reason TEXT,
  policy_result VARCHAR(32),
  created_at TIMESTAMPTZ DEFAULT now()
);

-- =====
-- 19. MESSAGE IDEMPOTENCY RETROFIT (v1.4 clause 63 – closes HIGH finding)
-- =====

ALTER TABLE agent_messages ADD COLUMN idempotency_key VARCHAR(128);
CREATE UNIQUE INDEX ux_agent_messages_tenant_idempotency
  ON agent_messages(tenant_id, idempotency_key);

-- =====
-- 20. INDEXES (clause 56 – every FK indexed, every tenant_id composite)
-- =====

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_membership_tenant_user ON organization_membership(tenant_id, user_id);
CREATE INDEX idx_roles_tenant ON roles(tenant_id);
CREATE INDEX idx_role_permissions_tenant_role ON role_permissions(tenant_id, role_id);
CREATE INDEX idx_user_roles_tenant_user ON user_roles(tenant_id, user_id);

CREATE INDEX idx_agent_registry_tenant_status ON agent_registry(tenant_id, status);
CREATE INDEX idx_agent_registry_tenant_lifecycle ON agent_registry(tenant_id, lifecycle_state);
CREATE INDEX idx_prompt_registry_tenant_agent ON prompt_registry(tenant_id, agent_id);

CREATE INDEX idx_policy_registry_tenant ON policy_registry(tenant_id, status);
CREATE INDEX idx_approval_requests_tenant_status ON approval_requests(tenant_id, decision);

CREATE INDEX idx_workflow_registry_tenant_status ON workflow_registry(tenant_id, status);
CREATE INDEX idx_workflow_history_tenant_workflow ON workflow_history(tenant_id, workflow_id, sequence_no);
CREATE INDEX idx_task_registry_tenant_status ON task_registry(tenant_id, status);
CREATE INDEX idx_task_registry_tenant_workflow ON task_registry(tenant_id, workflow_id);

CREATE INDEX idx_agent_messages_tenant_receiver ON agent_messages(tenant_id, receiver_agent_id, status);
CREATE INDEX idx_agent_messages_tenant_workflow ON agent_messages(tenant_id, workflow_id);

CREATE INDEX idx_working_memory_tenant_expiry ON working_memory_cache(tenant_id, expires_at);
CREATE INDEX idx_memory_facts_tenant_subject ON memory_facts(tenant_id, subject_type, subject_id);

CREATE INDEX idx_artifact_registry_tenant_project ON artifact_registry(tenant_id, project_id, status);

```

```

CREATE INDEX idx_artifact_registry_tenant_task ON artifact_registry(tenant_id, task_id);

CREATE INDEX idx_decision_records_tenant_project ON decision_records(tenant_id, project_id);
CREATE INDEX idx_audit_events_tenant_correlation ON audit_events(tenant_id, correlation_id);
CREATE INDEX idx_audit_events_tenant_created ON audit_events(tenant_id, created_at);

CREATE INDEX idx_secrets_refs_tenant ON secrets_vault_references(tenant_id, scope_agent_id);
CREATE INDEX idx_feature_flags_tenant_key ON feature_flags(tenant_id, flag_key, environment);
CREATE INDEX idx_config_versions_tenant_env ON configuration_versions(tenant_id, environment, created_at);

-- v1.4 new-table indexes
CREATE INDEX idx_project_registry_tenant_lifecycle ON project_registry(tenant_id, lifecycle_state);
CREATE INDEX idx_agent_versions_tenant_agent ON agent_versions(tenant_id, agent_id, version);
CREATE INDEX idx_agent_runs_tenant_task ON agent_runs(tenant_id, task_id);
CREATE INDEX idx_agent_runs_tenant_agent ON agent_runs(tenant_id, agent_id);
CREATE INDEX idx_model_runs_tenant_agentrun ON model_runs(tenant_id, agent_run_id);
CREATE INDEX idx_model_eval_runs_model ON model_evaluation_runs(model_id, executed_at);
CREATE INDEX idx_model_eval_metrics_eval ON model_evaluation_metrics(evaluation_id);
CREATE INDEX idx_usage_events_tenant_project ON usage_events(tenant_id, project_id, event_time);
CREATE INDEX idx_usage_events_tenant_agent ON usage_events(tenant_id, agent_id, event_time);
CREATE INDEX idx_release_registry_tenant_project ON release_registry(tenant_id, project_id);
CREATE INDEX idx_deployment_registry_tenant_project ON deployment_registry(tenant_id, project_id, status);
CREATE INDEX idx_tool_registry_mcp_server ON tool_registry(mcp_server_id);
CREATE INDEX idx_agent_tool_bindings_tenant_agent ON agent_tool_bindings(tenant_id, agent_id);
CREATE INDEX idx_policy_decisions_tenant_task ON policy_decision_records(tenant_id, task_id);
CREATE INDEX idx_routing_decisions_tenant_task ON routing_decision_records(tenant_id, task_id);

-- =====
-- 21. ROW-LEVEL SECURITY – FULL COVERAGE (v1.4 clause 62 – closes CRITICAL
-- finding: v1.3 enabled RLS on only 4 tables with USING only. v1.4 enables
-- RLS on EVERY tenant-scoped operational table, each with BOTH a USING
-- policy (governs visibility) and a WITH CHECK policy (governs what can be
-- inserted/updated) – USING alone would still let a service-layer bug
-- write a cross-tenant row even though it could never read it back.
--
-- Application must SET app.current_tenant_id per session/request. Tables
-- that legitimately allow NULL tenant_id (system templates: agent_registry,
-- roles, mcp_server_registry, feature_flags) permit NULL rows to be
-- visible/writable only as read-only templates at the application layer;
-- the policies below still require a real session tenant_id to write any
-- non-NULL-tenant row.
-- =====

-- Helper pattern used throughout: a table with a strictly non-null
-- tenant_id gets a straightforward equality policy on both USING and
-- WITH CHECK. A table that legitimately allows NULL tenant_id (system
-- templates) additionally permits NULL rows to be visible.

ALTER TABLE organizations ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_organizations ON organizations
  USING (tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE user_organization_membership ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_membership ON user_organization_membership
  USING (tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE roles ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_roles ON roles
  USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE role_permissions ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_role_permissions ON role_permissions
  USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE user_roles ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_user_roles ON user_roles
  USING (tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE agent_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_agent_registry ON agent_registry
  USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE agent_versions ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_agent_versions ON agent_versions
  USING (tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE prompt_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_prompt_registry ON prompt_registry
  USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE policy_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy_registry ON policy_registry
  USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

```

```

ALTER TABLE approval_requests ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_approval_requests ON approval_requests
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE project_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_project_registry ON project_registry
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE workflow_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_workflow_registry ON workflow_registry
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE workflow_history ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_workflow_history ON workflow_history
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE task_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_task_registry ON task_registry
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE agent_runs ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_agent_runs ON agent_runs
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE model_runs ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_model_runs ON model_runs
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE agent_messages ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_agent_messages ON agent_messages
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE a2a_capability_cards ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_a2a_cards ON a2a_capability_cards
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE working_memory_cache ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_working_memory ON working_memory_cache
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE memory_facts ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_memory_facts ON memory_facts
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE memory_embeddings ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_memory_embeddings ON memory_embeddings
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE artifact_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_artifact_registry ON artifact_registry
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE decision_records ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_decision_records ON decision_records
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE audit_events ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_audit_events ON audit_events
    USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE secrets_vault_references ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_secrets_refs ON secrets_vault_references
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE feature_flags ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_feature_flags ON feature_flags
    USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE configuration_versions ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_configuration_versions ON configuration_versions
    USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true))
    WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE usage_events ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_usage_events ON usage_events
    USING (tenant_id::text = current_setting('app.current_tenant_id', true))

```

```

WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE release_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_release_registry ON release_registry
  USING (tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE deployment_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_deployment_registry ON deployment_registry
  USING (tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE mcp_server_registry ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_mcp_server_registry ON mcp_server_registry
  USING (tenant_id IS NULL OR tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE agent_tool_bindings ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_agent_tool_bindings ON agent_tool_bindings
  USING (tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE policy_decision_records ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy_decisions ON policy_decision_records
  USING (tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

ALTER TABLE routing_decision_records ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_routing_decisions ON routing_decision_records
  USING (tenant_id::text = current_setting('app.current_tenant_id', true))
  WITH CHECK (tenant_id::text = current_setting('app.current_tenant_id', true));

-- Tables intentionally left WITHOUT tenant RLS (by design, not oversight):
-- users                -- a user row may belong to multiple tenants; the
--                       -- tenant boundary is enforced via
--                       -- user_organization_membership, not on users itself.
-- permissions          -- system-wide catalog by design (clause 41/44).
-- provider_registry,
-- model_registry,
-- model_evaluation_runs,
-- model_evaluation_metrics,
-- tool_registry        -- platform-wide catalogs; tenant-specific access
--                       -- is enforced at agent_tool_bindings /
--                       -- agent_registry.preferred_provider level, not here.
-- Every table storing tenant-owned operational DATA has RLS above.

```

Section D — Control Plane OpenAPI v1.4 (Full YAML — 40 Endpoints)

```

openapi: 3.0.3
info:
  title: AI Digital Office OS Control Plane
  version: 1.4.0
  description: >
    Governed control-plane API for AI DIGITAL OFFICE OS. Every endpoint is
    tenant-scoped via the JWT 'tenant_id' claim, versioned under /v1, and
    returns the standard error envelope on any non-2xx response (see
    ErrorResponse schema). v1.4 closes the integrity-review findings against
    v1.3: full project/agent-version/run/deployment/usage/tool lifecycle
    coverage, an explicit auth contract (issuer/audience/JWKS/scopes), and
    a required Idempotency-Key header on every state-changing endpoint.
    This specification is product-agnostic: no endpoint, schema, or example
    is specific to any named downstream project (see blueprint clause 75).
servers:
  - url: https://api.digitaloffice.example/v1
    description: Production (placeholder host – real endpoint is an
      environment-specific choice, intentionally not fabricated here)

security:
  - bearerAuth: []

components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
      description: >
        OAuth2 client-credentials token for service/agent callers, or
        OAuth2 authorization-code token for human users (blueprint clause 70).
        Required claims: 'iss' (issuer – this Office's or an approved
        external IdP), 'aud' (this control plane), 'exp'/'iat' (max lifetime
        15 minutes for service/agent callers, 8 hours for human sessions),
        'tenant_id', 'scope' (granted permission scopes), and 'actor_type'
        (SERVICE | HUMAN). Clock-skew tolerance ±60 seconds. JWKS rotates on
        a maximum 90-day cycle with a minimum 7-day overlap window. Session
        and refresh-token lifecycle is delegated entirely to the issuing IdP
        (blueprint clause 71) – this API only validates presented tokens.

parameters:

```

```
IdempotencyKey:
  name: Idempotency-Key
  in: header
  required: true
  schema: { type: string }
  description: >
    Required on every state-changing (POST/PATCH/DELETE) request.
    Replaying the same key within 24 hours returns the original result
    rather than reprocessing (blueprint clauses 49, 63, 68).
```

schemas:

```
ErrorResponse:
  type: object
  required: [error_code, message, correlation_id, retryable]
  properties:
    error_code:
      type: string
      enum: [VALIDATION_ERROR, AUTHORIZATION_ERROR, POLICY_ERROR, TOOL_ERROR,
        MODEL_ERROR, PROVIDER_ERROR, NETWORK_ERROR, TIMEOUT,
        DATABASE_ERROR, ARTIFACT_ERROR, DEPLOYMENT_ERROR, UNKNOWN_ERROR]
    message: { type: string }
    correlation_id: { type: string }
    retryable: { type: boolean }
```

Task:

```
  type: object
  required: [task_id, tenant_id, status]
  properties:
    task_id: { type: string }
    tenant_id: { type: string, format: uuid }
    project_id: { type: string }
    workflow_id: { type: string }
    parent_task_id: { type: string }
    assigned_agent: { type: string }
    required_capability: { type: string }
    priority: { type: string }
    risk_level: { type: string, enum: [GREEN, YELLOW, RED] }
    security_level: { type: string }
    dependencies: { type: array, items: { type: string } }
    input: { type: object }
    expected_output: { type: object }
    status:
      type: string
      enum: [CREATED, QUEUED, ASSIGNED, RUNNING, WAITING, BLOCKED, FAILED, RETRYING, COMPLETED, CANCELLED, ESCALATED]
    retry_count: { type: integer }
    idempotency_key: { type: string }
    deadline: { type: string, format: date-time }
```

TaskCreateRequest:

```
  type: object
  required: [project_id, required_capability, input, idempotency_key]
  properties:
    project_id: { type: string }
    workflow_id: { type: string }
    parent_task_id: { type: string }
    required_capability: { type: string }
    priority: { type: string, enum: [LOW, NORMAL, HIGH, CRITICAL] }
    risk_level: { type: string, enum: [GREEN, YELLOW, RED] }
    security_level: { type: string }
    dependencies: { type: array, items: { type: string } }
    input: { type: object }
    expected_output: { type: object }
    idempotency_key:
      type: string
      description: Required – guarantees a retried request never double-executes.
    deadline: { type: string, format: date-time }
```

AgentSpecification:

```
  type: object
  required: [agent_name, department, role, purpose, capabilities]
  properties:
    agent_name: { type: string }
    department: { type: string }
    role: { type: string }
    purpose: { type: string }
    capabilities: { type: array, items: { type: string } }
    required_model_capabilities: { type: array, items: { type: string } }
    allowed_tools: { type: array, items: { type: string } }
    permissions: { type: array, items: { type: string } }
    data_access: { type: array, items: { type: string } }
    input_schema: { type: object }
    output_schema: { type: object }
    security_level: { type: string, enum: [GREEN, YELLOW, RED], default: GREEN }
    sandbox_policy: { type: object }
    parent_agent_id: { type: string }
```

AgentRecord:

```
  allOf:
    - $ref: '#/components/schemas/AgentSpecification'
    - type: object
      properties:
        agent_id: { type: string }
        tenant_id: { type: string, format: uuid, nullable: true }
```

```

    lifecycle_state:
      type: string
      enum: [DRAFT, SANDBOX, TESTED, EVALUATED, APPROVED, ACTIVE, UPDATED, DEPRECATED, RETIRED]
    evaluation_score: { type: number }
    success_rate: { type: number }
    average_latency_ms: { type: integer }
    average_cost: { type: number }
    version: { type: string }
    parent_agent_id: { type: string, nullable: true }
    active_agent_version_id: { type: string, format: uuid, nullable: true, description: "v1.4 – points at the
currently ACTIVE agent_versions row (clause 60)." }
    preferred_provider: { type: string }
    preferred_model: { type: string }
    fallback_models: { type: array, items: { type: string } }
    status: { type: string }
    created_at: { type: string, format: date-time }
    updated_at: { type: string, format: date-time }
    description: "v1.4: field set now matches agent_registry columns 1:1 – closes the v1.3 schema/SQL drift finding
(clause 69)."
```

```

RoutingRequest:
  type: object
  required: [task_id, agent_id, required_capability]
  properties:
    task_id: { type: string }
    agent_id: { type: string }
    required_capability: { type: string }
    security_classification: { type: string }
    data_classification: { type: string }
    max_cost: { type: number }
    max_latency_ms: { type: integer }
```

```

RoutingDecision:
  type: object
  properties:
    task_id: { type: string }
    agent_id: { type: string }
    selected_provider: { type: string }
    selected_model: { type: string }
    reason: { type: string }
    policy_result: { type: string, enum: [ALLOW, DENY, REQUIRE_APPROVAL, REQUIRE_ESCALATION] }
    fallback_chain_used: { type: array, items: { type: string } }
    decided_at: { type: string, format: date-time }
```

```

ApprovalRequest:
  type: object
  required: [action, risk_level]
  properties:
    task_id: { type: string }
    agent_id: { type: string }
    action:
      type: string
      description: "Includes AGENT_ACTIVATE for the Factory-Production transition (blueprint clause 45)."
```

```

    risk_level: { type: string, enum: [GREEN, YELLOW, RED] }
    reason: { type: string }
```

```

ApprovalRecord:
  allOf:
    - $ref: '#/components/schemas/ApprovalRequest'
    - type: object
      properties:
        request_id: { type: string }
        tenant_id: { type: string, format: uuid }
        requester: { type: string }
        approver: { type: string }
        decision: { type: string, enum: [APPROVED, DENIED, EXPIRED] }
        expires_at: { type: string, format: date-time }
        execution_result: { type: object }
        created_at: { type: string, format: date-time }
        decided_at: { type: string, format: date-time }
```

```

Workflow:
  type: object
  properties:
    workflow_id: { type: string }
    tenant_id: { type: string, format: uuid }
    project_id: { type: string }
    workflow_type: { type: string }
    definition_version: { type: string }
    current_state: { type: object }
    status: { type: string, enum: [RUNNING, PAUSED, COMPLETED, FAILED, CANCELLED, ESCALATED] }
    started_at: { type: string, format: date-time }
    completed_at: { type: string, format: date-time }
```

```

Artifact:
  type: object
  properties:
    artifact_id: { type: string, format: uuid }
    tenant_id: { type: string, format: uuid }
    project_id: { type: string }
    task_id: { type: string }
    artifact_type: { type: string, enum: [CODE, DOC, DESIGN, CONFIG, REPORT, DATASET] }
    storage_uri: { type: string }
```

```

    content_hash: { type: string }
    version: { type: string }
    status: { type: string, enum: [DRAFT, IN_REVIEW, APPROVED, RELEASED, DEPRECATED] }
    parent_artifact_id: { type: string, format: uuid, nullable: true }
    agent_run_id: { type: string, nullable: true, description: "v1.4 – resolvable FK into agent_runs (clause 61)." }
    model_run_id: { type: string, nullable: true, description: "v1.4 – resolvable FK into model_runs (clause 61)." }
    reviewed_by: { type: string }
    approved_by: { type: string }
    git_commit_ref: { type: string }
    deployment_ref: { type: string, deprecated: true, description: "Superseded by /deployments and deployment_register (clause 66); retained read-only for backward compatibility." }
    created_at: { type: string, format: date-time }

MemoryQueryRequest:
  type: object
  required: [query_text]
  properties:
    query_text: { type: string }
    scope: { type: string, enum: [AGENT, PROJECT, ORG] }
    subject_id: { type: string }
    top_k: { type: integer, default: 10 }
    distance_threshold: { type: number, default: 0.4 }

MemoryQueryResult:
  type: object
  properties:
    embedding_id: { type: string, format: uuid }
    content: { type: string }
    similarity: { type: number }
    memory_fact_id: { type: string, format: uuid, nullable: true }

FeatureFlag:
  type: object
  properties:
    flag_id: { type: string, format: uuid }
    flag_key: { type: string }
    flag_type: { type: string, enum: [BOOLEAN, PERCENTAGE, VARIANT] }
    default_value: {}
    tenant_override_value: {}
    environment: { type: string }
    status: { type: string }
    tenant_id: { type: string, format: uuid, nullable: true, description: "NULL = global default (clause 51)." }
    created_by: { type: string }
    updated_at: { type: string, format: date-time }

Tenant:
  type: object
  properties:
    tenant_id: { type: string, format: uuid }
    org_name: { type: string }
    org_slug: { type: string }
    status: { type: string, enum: [ACTIVE, SUSPENDED, ARCHIVED] }
    plan_tier: { type: string }

# ---- New in v1.4 ----

Project:
  type: object
  required: [project_name, project_type]
  properties:
    project_id: { type: string }
    tenant_id: { type: string, format: uuid }
    project_name: { type: string }
    project_type:
      type: string
      description: "Free-form. No product type receives special-cased treatment (blueprint clause 75)."
    constitution_version: { type: string }
    lifecycle_state:
      type: string
      enum: [DISCOVERY, RESEARCH, REQUIREMENTS, PRODUCT_PLAN, ARCHITECTURE, UX_UI, DESIGN, DEVELOPMENT, CODE_REVIEW, QA, SECURITY, STAGING, APPROVAL, DEPLOYMENT, MONITORING, MAINTENANCE]
    risk_level: { type: string, enum: [GREEN, YELLOW, RED] }
    owner_user_id: { type: string, format: uuid }
    repository_ref: { type: string }
    environment_policy: { type: object }

AgentVersion:
  type: object
  properties:
    agent_version_id: { type: string, format: uuid }
    agent_id: { type: string }
    version: { type: string }
    specification_hash: { type: string }
    prompt_version: { type: string }
    model_policy: { type: object }
    permissions_snapshot: { type: object }
    evaluation_score: { type: number }
    lifecycle_state: { type: string }

ArtifactCreateRequest:
  type: object
  required: [project_id, artifact_type, storage_uri, content_hash]
  properties:

```

```

    project_id: { type: string }
    task_id: { type: string }
    agent_run_id: { type: string }
    model_run_id: { type: string }
    artifact_type: { type: string, enum: [CODE, DOC, DESIGN, CONFIG, REPORT, DATASET] }
    storage_uri: { type: string }
    content_hash: { type: string }
    version: { type: string, default: "1.0.0" }

ArtifactLineage:
  type: object
  properties:
    artifact: { $ref: '#/components/schemas/Artifact' }
    agent_run: { type: object }
    model_run: { type: object }
    task: { $ref: '#/components/schemas/Task' }
    workflow: { $ref: '#/components/schemas/Workflow' }
    parent_chain: { type: array, items: { $ref: '#/components/schemas/Artifact' } }

ModelEvaluationTriggerRequest:
  type: object
  required: [model_id, benchmark_suite]
  properties:
    model_id: { type: string }
    benchmark_suite: { type: string }
    evaluator_version: { type: string }

ModelEvaluationResult:
  type: object
  properties:
    evaluation_id: { type: string, format: uuid }
    model_id: { type: string }
    benchmark_suite: { type: string }
    score: { type: number }
    metrics:
      type: array
      items:
        type: object
        properties:
          metric_name: { type: string }
          metric_value: { type: number }
          unit: { type: string }
    executed_at: { type: string, format: date-time }

UsageEvent:
  type: object
  properties:
    usage_event_id: { type: string, format: uuid }
    project_id: { type: string }
    task_id: { type: string }
    agent_id: { type: string }
    provider_id: { type: string }
    model_id: { type: string }
    input_tokens: { type: integer }
    output_tokens: { type: integer }
    actual_cost: { type: number }
    currency: { type: string }
    event_time: { type: string, format: date-time }

CostSummary:
  type: object
  properties:
    tenant_id: { type: string, format: uuid }
    period_start: { type: string, format: date-time }
    period_end: { type: string, format: date-time }
    total_cost: { type: number }
    currency: { type: string }
    budget_status: { type: string, enum: [OK, WARNING, SOFT_LIMIT, HARD_LIMIT] }
    breakdown_by_provider: { type: object }

ToolRecord:
  type: object
  properties:
    tool_id: { type: string }
    mcp_server_id: { type: string }
    tool_name: { type: string }
    version: { type: string }
    risk_level: { type: string, enum: [GREEN, YELLOW, RED] }
    enabled: { type: boolean }

McpServerRecord:
  type: object
  properties:
    mcp_server_id: { type: string }
    server_name: { type: string }
    endpoint: { type: string }
    trust_level: { type: string }
    enabled: { type: boolean }

DeploymentCreateRequest:
  type: object
  required: [project_id, release_id, environment, strategy]
  properties:

```

```

    project_id: { type: string }
    release_id: { type: string }
    environment: { type: string }
    strategy: { type: string, enum: [standard, rolling, blue_green, canary] }
    approval_request_id: { type: string }

Deployment:
  allOf:
    - $ref: '#/components/schemas/DeploymentCreateRequest'
    - type: object
  properties:
    deployment_id: { type: string }
    tenant_id: { type: string, format: uuid }
    status: { type: string }
    rollback_target: { type: string }
    started_at: { type: string, format: date-time }
    completed_at: { type: string, format: date-time }

PolicyDecisionRecord:
  type: object
  properties:
    policy_decision_id: { type: string, format: uuid }
    task_id: { type: string }
    agent_id: { type: string }
    decision: { type: string, enum: [ALLOW, DENY, REQUIRE_APPROVAL, REQUIRE_ESCALATION] }
    policy_version: { type: string }
    alternatives: { type: object }
    rejection_reasons: { type: object }
    created_at: { type: string, format: date-time }

RoutingDecisionRecord:
  type: object
  properties:
    routing_decision_id: { type: string, format: uuid }
    task_id: { type: string }
    selected_provider: { type: string }
    selected_model: { type: string }
    candidate_models: { type: object }
    reason: { type: string }
    created_at: { type: string, format: date-time }

responses:
  Unauthorized:
    description: Missing/invalid bearer token
    content: { application/json: { schema: { $ref: '#/components/schemas/ErrorResponse' } } }
  PolicyDenied:
    description: Central Policy Engine returned DENY
    content: { application/json: { schema: { $ref: '#/components/schemas/ErrorResponse' } } }
  NotFound:
    description: Resource not found (or not visible to this tenant)
    content: { application/json: { schema: { $ref: '#/components/schemas/ErrorResponse' } } }

paths:
  /tasks:
    post:
      summary: Create a governed task
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/TaskCreateRequest' }
      responses:
        '201':
          description: Task created
          content: { application/json: { schema: { $ref: '#/components/schemas/Task' } } }
        '403': { $ref: '#/components/responses/PolicyDenied' }
        '401': { $ref: '#/components/responses/Unauthorized' }

  /tasks/{task_id}:
    get:
      summary: Get task state
      parameters:
        - { name: task_id, in: path, required: true, schema: { type: string } }
      responses:
        '200':
          description: Task state
          content: { application/json: { schema: { $ref: '#/components/schemas/Task' } } }
        '404': { $ref: '#/components/responses/NotFound' }

  /agents:
    post:
      summary: Submit an agent specification to Agent Factory
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/AgentSpecification' }
      responses:
        '202':
          description: Agent creation workflow started (lands in DRAFT, not ACTIVE)
          content: { application/json: { schema: { $ref: '#/components/schemas/AgentRecord' } } }
        '403': { $ref: '#/components/responses/PolicyDenied' }
    get:

```

```

summary: List agents visible to the caller's tenant
parameters:
  - { name: lifecycle_state, in: query, schema: { type: string } }
  - { name: department, in: query, schema: { type: string } }
responses:
  '200':
    description: Agent list
    content:
      application/json:
        schema: { type: array, items: { $ref: '#/components/schemas/AgentRecord' } }

/agents/{agent_id}:
get:
  summary: Get agent registry record
  parameters:
    - { name: agent_id, in: path, required: true, schema: { type: string } }
  responses:
    '200':
      description: Agent record
      content: { application/json: { schema: { $ref: '#/components/schemas/AgentRecord' } } }
    '404': { $ref: '#/components/responses/NotFound' }

/agents/{agent_id}/messages:
get:
  summary: List agent-to-agent messages sent to/from this agent
  parameters:
    - { name: agent_id, in: path, required: true, schema: { type: string } }
    - { name: status, in: query, schema: { type: string } }
  responses:
    '200':
      description: Message list
      content: { application/json: { schema: { type: array, items: { type: object } } } }

/models/route:
post:
  summary: Request capability-based model routing
  requestBody:
    required: true
    content:
      application/json:
        schema: { $ref: '#/components/schemas/RoutingRequest' }
  responses:
    '200':
      description: Routing decision
      content: { application/json: { schema: { $ref: '#/components/schemas/RoutingDecision' } } }
    '403': { $ref: '#/components/responses/PolicyDenied' }

/approvals:
post:
  summary: Create an approval request (includes AGENT_ACTIVATE)
  requestBody:
    required: true
    content:
      application/json:
        schema: { $ref: '#/components/schemas/ApprovalRequest' }
  responses:
    '201':
      description: Approval request created
      content: { application/json: { schema: { $ref: '#/components/schemas/ApprovalRecord' } } }
get:
  summary: List pending approvals for the caller's tenant
  parameters:
    - { name: risk_level, in: query, schema: { type: string, enum: [GREEN, YELLOW, RED] } }
  responses:
    '200':
      description: Approval list
      content:
        application/json:
          schema: { type: array, items: { $ref: '#/components/schemas/ApprovalRecord' } }

/approvals/{request_id}/decision:
post:
  summary: Record an approval decision (management/Owner action)
  parameters:
    - { name: request_id, in: path, required: true, schema: { type: string } }
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          required: [decision]
          properties:
            decision: { type: string, enum: [APPROVED, DENIED] }
            reason: { type: string }
  responses:
    '200':
      description: Updated approval record
      content: { application/json: { schema: { $ref: '#/components/schemas/ApprovalRecord' } } }

/providers/health:
get:
  summary: Provider health status

```

```

    responses:
      '200':
        description: Provider health
        content: { application/json: { schema: { type: array, items: { type: object } } } }

/workflows/{workflow_id}:
  get:
    summary: Get durable workflow state (reconstructed from workflow_history on demand)
    parameters:
      - { name: workflow_id, in: path, required: true, schema: { type: string } }
    responses:
      '200':
        description: Workflow state
        content: { application/json: { schema: { $ref: '#/components/schemas/Workflow' } } }
      '404': { $ref: '#/components/responses/NotFound' }

/workflows/{workflow_id}/pause:
  post:
    summary: Pause a running workflow
    parameters:
      - { name: workflow_id, in: path, required: true, schema: { type: string } }
    responses:
      '200': { description: Workflow paused }

/workflows/{workflow_id}/resume:
  post:
    summary: Resume a paused workflow
    parameters:
      - { name: workflow_id, in: path, required: true, schema: { type: string } }
    responses:
      '200': { description: Workflow resumed }

/artifacts:
  get:
    summary: List artifacts for a project
    parameters:
      - { name: project_id, in: query, required: true, schema: { type: string } }
      - { name: status, in: query, schema: { type: string } }
    responses:
      '200':
        description: Artifact list
        content:
          application/json:
            schema: { type: array, items: { $ref: '#/components/schemas/Artifact' } }

/artifacts/{artifact_id}:
  get:
    summary: Get artifact record and lineage
    parameters:
      - { name: artifact_id, in: path, required: true, schema: { type: string, format: uuid } }
    responses:
      '200':
        description: Artifact record
        content: { application/json: { schema: { $ref: '#/components/schemas/Artifact' } } }
      '404': { $ref: '#/components/responses/NotFound' }

/memory/query:
  post:
    summary: Tiered memory retrieval (routes to working/durable/semantic tier)
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: '#/components/schemas/MemoryQueryRequest' }
    responses:
      '200':
        description: Ranked memory results
        content:
          application/json:
            schema: { type: array, items: { $ref: '#/components/schemas/MemoryQueryResult' } }

/feature-flags:
  get:
    summary: List feature flags visible to the caller's tenant
    responses:
      '200':
        description: Flag list
        content:
          application/json:
            schema: { type: array, items: { $ref: '#/components/schemas/FeatureFlag' } }
  post:
    summary: Create or override a feature flag for this tenant
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: '#/components/schemas/FeatureFlag' }
    responses:
      '201':
        description: Flag created/updated
        content: { application/json: { schema: { $ref: '#/components/schemas/FeatureFlag' } } }

/tenants/current:

```

```

get:
  summary: Get the calling token's own tenant record
  responses:
    '200':
      description: Tenant record
      content: { application/json: { schema: { $ref: '#/components/schemas/Tenant' } } }

/secrets/{reference_id}/rotate:
  post:
    summary: Trigger rotation of a referenced secret (never returns the raw value)
    parameters:
      - { name: reference_id, in: path, required: true, schema: { type: string, format: uuid } }
      - $ref: '#/components/parameters/IdempotencyKey'
    responses:
      '202': { description: Rotation triggered – audited as a secret-access event }
      '403': { $ref: '#/components/responses/PolicyDenied' }

# ===== New in v1.4 – closes the "incomplete API lifecycle" finding =====

/projects:
  post:
    summary: Create a project (authoritative project_registry record)
    parameters: [ { $ref: '#/components/parameters/IdempotencyKey' } ]
    requestBody:
      required: true
      content: { application/json: { schema: { $ref: '#/components/schemas/Project' } } }
    responses:
      '201': { description: Project created, content: { application/json: { schema: { $ref:
'#/components/schemas/Project' } } } }
  get:
    summary: List projects for the caller's tenant
    parameters:
      - { name: lifecycle_state, in: query, schema: { type: string } }
    responses:
      '200': { description: Project list, content: { application/json: { schema: { type: array, items: { $ref:
'#/components/schemas/Project' } } } } }

/projects/{project_id}:
  get:
    summary: Get a project record
    parameters: [ { name: project_id, in: path, required: true, schema: { type: string } } ]
    responses:
      '200': { description: Project record, content: { application/json: { schema: { $ref:
'#/components/schemas/Project' } } } }
      '404': { $ref: '#/components/responses/NotFound' }
  patch:
    summary: Update a project's mutable fields (lifecycle_state, risk_level, environment_policy, etc.)
    parameters: [ { name: project_id, in: path, required: true, schema: { type: string } } ]
    requestBody:
      required: true
      content: { application/json: { schema: { type: object } } }
    responses:
      '200': { description: Updated project, content: { application/json: { schema: { $ref:
'#/components/schemas/Project' } } } }

/workflows:
  post:
    summary: Start a new durable workflow
    parameters: [ { $ref: '#/components/parameters/IdempotencyKey' } ]
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [project_id, workflow_type, definition_version]
            properties:
              project_id: { type: string }
              workflow_type: { type: string }
              definition_version: { type: string }
    responses:
      '201': { description: Workflow started, content: { application/json: { schema: { $ref:
'#/components/schemas/Workflow' } } } }

/workflows/{workflow_id}/cancel:
  post:
    summary: Cancel a workflow (writes a workflow_history CANCELLED event)
    parameters: [ { name: workflow_id, in: path, required: true, schema: { type: string } } ]
    responses: { '200': { description: Workflow cancelled } }

/workflows/{workflow_id}/retry:
  post:
    summary: Retry the current failed step of a workflow
    parameters: [ { name: workflow_id, in: path, required: true, schema: { type: string } } ]
    responses: { '200': { description: Retry initiated } }

/workflows/{workflow_id}/escalate:
  post:
    summary: Escalate a workflow to human review (creates an approval_requests row)
    parameters: [ { name: workflow_id, in: path, required: true, schema: { type: string } } ]
    responses: { '200': { description: Escalation created, content: { application/json: { schema: { $ref:
'#/components/schemas/ApprovalRecord' } } } } }

```

```

/agents/{agent_id}/versions:
  post:
    summary: Submit a new agent version (does not activate it)
    parameters:
      - { name: agent_id, in: path, required: true, schema: { type: string } }
      - $ref: '#/components/parameters/IdempotencyKey'
    requestBody:
      required: true
      content: { application/json: { schema: { $ref: '#/components/schemas/AgentVersion' } } }
    responses:
      '201': { description: Version created, content: { application/json: { schema: { $ref:
'#/components/schemas/AgentVersion' } } } }
  get:
    summary: List all versions of an agent
    parameters: [ { name: agent_id, in: path, required: true, schema: { type: string } } ]
    responses:
      '200': { description: Version list, content: { application/json: { schema: { type: array, items: { $ref:
'#/components/schemas/AgentVersion' } } } } }

/agents/{agent_id}/versions/{agent_version_id}/activate:
  post:
    summary: Activate a specific agent version into production (requires prior AGENT_ACTIVATE approval – clause 45/60)
    parameters:
      - { name: agent_id, in: path, required: true, schema: { type: string } }
      - { name: agent_version_id, in: path, required: true, schema: { type: string } }
    responses:
      '200': { description: Version activated }
      '403': { $ref: '#/components/responses/PolicyDenied' }

/agents/{agent_id}/messages:
  post:
    summary: Send a typed agent-to-agent message (idempotency-keyed – clause 63)
    parameters:
      - { name: agent_id, in: path, required: true, schema: { type: string } }
      - $ref: '#/components/parameters/IdempotencyKey'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [receiver_agent_id, message_type]
            properties:
              receiver_agent_id: { type: string }
              message_type: { type: string }
              input_payload: { type: object }
    responses:
      '202': { description: Message queued for delivery }

/artifacts:
  post:
    summary: Register a new artifact (immutable once status=APPROVED – clause 48)
    parameters: [ { $ref: '#/components/parameters/IdempotencyKey' } ]
    requestBody:
      required: true
      content: { application/json: { schema: { $ref: '#/components/schemas/ArtifactCreateRequest' } } }
    responses:
      '201': { description: Artifact created, content: { application/json: { schema: { $ref:
'#/components/schemas/Artifact' } } } }

/artifacts/{artifact_id}/lineage:
  get:
    summary: Resolve full lineage for an artifact (Owner Command → ... → Release – clause 23/61)
    parameters: [ { name: artifact_id, in: path, required: true, schema: { type: string, format: uuid } } ]
    responses:
      '200': { description: Full lineage, content: { application/json: { schema: { $ref:
'#/components/schemas/ArtifactLineage' } } } }
      '404': { $ref: '#/components/responses/NotFound' }

/providers:
  get:
    summary: List registered AI providers and health status
    responses:
      '200': { description: Provider list, content: { application/json: { schema: { type: array, items: { type: object
} } } } }

/models:
  get:
    summary: List registered models with capability tags
    parameters:
      - { name: capability, in: query, schema: { type: string } }
    responses:
      '200': { description: Model list, content: { application/json: { schema: { type: array, items: { type: object } }
} } }

/models/evaluate:
  post:
    summary: Trigger a model evaluation run
    parameters: [ { $ref: '#/components/parameters/IdempotencyKey' } ]
    requestBody:
      required: true
      content: { application/json: { schema: { $ref: '#/components/schemas/ModelEvaluationTriggerRequest' } } }
    responses:

```

```

      '202': { description: Evaluation triggered, content: { application/json: { schema: { $ref:
'#/components/schemas/ModelEvaluationResult' } } } }

/models/evaluations:
  get:
    summary: List historical model evaluation runs (clause 64)
    parameters:
      - { name: model_id, in: query, schema: { type: string } }
    responses:
      '200': { description: Evaluation history, content: { application/json: { schema: { type: array, items: { $ref:
'#/components/schemas/ModelEvaluationResult' } } } } }

/tools:
  get:
    summary: List MCP-registered tools available to the caller's tenant
    responses:
      '200': { description: Tool list, content: { application/json: { schema: { type: array, items: { $ref:
'#/components/schemas/ToolRecord' } } } } }

/mcp/servers:
  get:
    summary: List registered MCP servers
    responses:
      '200': { description: Server list, content: { application/json: { schema: { type: array, items: { $ref:
'#/components/schemas/McpServerRecord' } } } } }

/usage:
  get:
    summary: Query raw usage events (clause 65)
    parameters:
      - { name: project_id, in: query, schema: { type: string } }
      - { name: agent_id, in: query, schema: { type: string } }
      - { name: from, in: query, schema: { type: string, format: date-time } }
      - { name: to, in: query, schema: { type: string, format: date-time } }
    responses:
      '200': { description: Usage events, content: { application/json: { schema: { type: array, items: { $ref:
'#/components/schemas/UsageEvent' } } } } }

/costs:
  get:
    summary: Aggregated cost summary against tenant budget tiers (clause 30/65)
    parameters:
      - { name: from, in: query, schema: { type: string, format: date-time } }
      - { name: to, in: query, schema: { type: string, format: date-time } }
    responses:
      '200': { description: Cost summary, content: { application/json: { schema: { $ref:
'#/components/schemas/CostSummary' } } } }

/deployments:
  post:
    summary: Create a deployment for a release (clause 66)
    parameters: [ { $ref: '#/components/parameters/IdempotencyKey' } ]
    requestBody:
      required: true
    content: { application/json: { schema: { $ref: '#/components/schemas/DeploymentCreateRequest' } } }
    responses:
      '201': { description: Deployment created, content: { application/json: { schema: { $ref:
'#/components/schemas/Deployment' } } } }

/deployments/{deployment_id}:
  get:
    summary: Get deployment status
    parameters: [ { name: deployment_id, in: path, required: true, schema: { type: string } } ]
    responses:
      '200': { description: Deployment record, content: { application/json: { schema: { $ref:
'#/components/schemas/Deployment' } } } }
      '404': { $ref: '#/components/responses/NotFound' }

/deployments/{deployment_id}/rollback:
  post:
    summary: Roll back a deployment to its recorded rollback_target
    parameters:
      - { name: deployment_id, in: path, required: true, schema: { type: string } }
      - $ref: '#/components/parameters/IdempotencyKey'
    responses:
      '202': { description: Rollback triggered }

/policy-decisions:
  get:
    summary: Query structured policy decision records (clause 72)
    parameters:
      - { name: task_id, in: query, schema: { type: string } }
      - { name: agent_id, in: query, schema: { type: string } }
    responses:
      '200': { description: Policy decisions, content: { application/json: { schema: { type: array, items: { $ref:
'#/components/schemas/PolicyDecisionRecord' } } } } }

/routing-decisions:
  get:
    summary: Query structured routing decision records (clause 72)
    parameters:
      - { name: task_id, in: query, schema: { type: string } }
    responses:

```

```
'200': { description: Routing decisions, content: { application/json: { schema: { type: array, items: { $ref:
'#/components/schemas/RoutingDecisionRecord' } } } } }
```

Section E — Architecture & Sequence Diagrams v1.4

AI DIGITAL OFFICE OS v1.4 Architecture Diagrams

All three v1.2 diagrams are preserved unchanged below, followed by four new sequence/structure diagrams that close the “no sequence diagrams” gap flagged in the v1.2 review.

AI DIGITAL OFFICE OS v1.2/v1.3/v1.4 Architecture (unchanged since v1.2, still authoritative)

```
flowchart TD
    O[Owner / Chairman] --> CEO[Master CEO]
    CEO --> EM[Executive Manager]
    EM --> AM[Assistant Manager]
    AM --> CP[Orchestration Control Plane]

    CP --> AF[Agent Factory]
    AF --> AR[Agent Registry]
    AF --> SB[Sandbox]
    AF --> EV[Agent Evaluation]
    AR --> AP[Specialized Agent Pool]

    AP --> PE[Central Policy Engine]
    PE --> AUTH[RBAC / ABAC / Permissions]

    AP --> MR[Model Router]
    MR --> MREG[Model Registry]
    MREG --> PG[Provider Gateway]
    PG --> PA[Provider Adapters]
    PA --> P1[OpenAI/GPT]
    PA --> P2[Anthropic/Claude]
    PA --> P3[Google/Gemini]
    PA --> P4[Moonshot/Kimi]
    PA --> P5[Local Models]
    PA --> PF[Future Providers]

    AP --> TG[MCP / Tool Gateway]
    TG --> G[Git]
    TG --> D[Docker]
    TG --> DB[Postgres]
    TG --> FS[Files]
    TG --> WEB[Browsers / Web]
    TG --> AUTO[Automation / n8n]

    CP --> WM[Workflow + Task Engine]
    WM --> DAG[Dependency DAG]

    AP --> MEM[Memory / Knowledge]
    MEM --> V[pgvector]
    AP --> ART[Artifact Registry]
    ART --> GIT[Version Control]

    CP --> QA[Independent QA]
    CP --> SEC[Security]
    CP --> OBS[Observability]
    CP --> COST[Cost Governance]

    AP --> PFAC[Project Factory]
    PFAC --> DEV[Development]
    PFAC --> TEST[Testing]
    PFAC --> REL[Release]
    REL --> MON[Monitoring]

    PE --> APP[Approval Engine]
    APP --> O
```

Agent Factory lifecycle (unchanged from v1.2)

```
flowchart LR
    R[Request] --> S[Specification]
    S --> B[Sandbox]
    B --> T[Tests]
    T --> E[Evaluation]
    E --> SEC[Security]
    SEC --> A[Approval]
    A --> REG[Registry]
    REG --> ACT[Active]
    ACT --> U[Update]
    U --> E
```

```
ACT --> DEP[Deprecate]
DEP --> RET[Retire]
```

Software Factory lifecycle (unchanged from v1.2)

```
flowchart LR
I[Idea] --> R[Research]
R --> PRD[Requirements / PRD]
PRD --> ARC[Architecture]
ARC --> UX[UI/UX]
UX --> DEV[Build]
DEV --> REV[Code Review]
REV --> QA[QA]
QA --> SEC[Security]
SEC --> STG[Staging]
STG --> APP[Approval]
APP --> PROD[Production]
PROD --> OBS[Monitoring]
OBS --> MAINT[Maintenance]
MAINT --> I
```

NEW IN v1.3 — Sequence & Structure Diagrams

1. Task Routing Sequence (clauses 10, 17, 43, 49)

```
sequenceDiagram
    participant T as Task Engine
    participant PE as Policy Engine
    participant MR as Model Router
    participant MREG as Model Registry
    participant PG as Provider Gateway
    participant P as AI Provider

    T->>PE: Evaluate(agent, task, data_class, risk)
    PE-->>T: ALLOW | DENY | REQUIRE_APPROVAL
    alt ALLOW
        T->>MR: Route(required_capability, policy_result)
        MR->>MREG: Lookup candidate models by capability
        MREG-->>MR: Ranked candidates (health, cost, latency)
        MR->>PG: Invoke selected provider/model
        PG->>P: Request (scoped short-lived credential)
        alt Provider healthy
            P-->>PG: Response
            PG-->>MR: Result
        else Provider failure / timeout
            PG-->>MR: Error (retryable)
            MR->>MR: Bounded backoff (base 500ms, x2, max 5 attempts)
            MR->>PG: Try Approved Secondary
            PG->>P: Request (fallback provider)
            P-->>PG: Response
            PG-->>MR: Result
        end
        MR-->>T: Routing decision + result (fully audited)
    else DENY
        T-->>T: Task marked FAILED, error_code=POLICY_ERROR
    else REQUIRE_APPROVAL
        T->>PE: Create approval requests row
        PE-->>T: Task status = WAITING
    end
end
```

2. Agent Creation → Activation Sequence (clauses 4, 5, 14, 45)

```
sequenceDiagram
    participant U as Requester (human or governed agent)
    participant AF as Agent Factory
    participant SB as Sandbox
    participant EV as Evaluation Engine
    participant SEC as Security Review
    participant APR as Approval Engine
    participant AR as Agent Registry

    U->>AF: Submit AgentSpecification
    AF->>AF: lifecycle_state = DRAFT
    AF->>SB: Deploy to sandbox (no prod credentials)
    SB-->>AF: lifecycle_state = SANDBOX
    AF->>EV: Run evaluation suite
    EV-->>AF: lifecycle_state = TESTED / EVALUATED (score recorded)
    AF->>SEC: Security review
    SEC-->>AF: Pass/Fail
    AF->>APR: Request approval (action=AGENT_ACTIVATE, risk=per policy)
    Note over APR: Factory CANNOT self-activate – clause 45
```

```
APR-->>AF: lifecycle_state = APPROVED (if approved)
APR-->>AR: On explicit activation trigger: lifecycle_state = ACTIVE
AR-->>U: Agent is now callable in production
```

3. Approval / Risk-Tier Escalation Flow (clause 14)

```
sequenceDiagram
    participant A as Requesting Agent
    participant PE as Policy Engine
    participant APR as Approval Engine
    participant MGR as Designated Manager (YELLOW)
    participant OWN as Owner/Chairman (RED)

    A->>PE: Attempt action
    PE-->>A: Risk classification (GREEN | YELLOW | RED)
    alt GREEN
        PE-->>A: Autonomous execution permitted
    else YELLOW
        PE-->>APR: Create approval_requests row
        APR-->>MGR: Route for management review
        MGR-->>APR: Decision (APPROVED/DENIED)
        APR-->>A: Execute or block per decision
    else RED
        PE-->>APR: Create approval_requests row
        APR-->>OWN: Route for Owner sign-off
        OWN-->>APR: Decision (APPROVED/DENIED)
        APR-->>A: Execute or block per decision
    end
    end
    APR-->>APR: Write audit_events (full trail, unlimited retention for RED)
```

4. Multi-Tenant Isolation (structural, clause 41/44)

```
flowchart TD
    subgraph Tenant A
        UA[Users A] --> MA[Membership A]
        MA --> RA[Roles A]
        RA --> RPA[role_permissions A]
        UA --> TAA[task_registry rows: tenant_id=A]
        UA --> AGA[agent_registry rows: tenant_id=A]
    end

    subgraph Tenant B
        UB[Users B] --> MB[Membership B]
        MB --> RB[Roles B]
        RB --> RPB[role_permissions B]
        UB --> TAB[task_registry rows: tenant_id=B]
        UB --> AGB[agent_registry rows: tenant_id=B]
    end

    PERM[permissions catalog – system-wide, tenant-agnostic]
    RPA --> PERM
    RPB --> PERM

    RLS[Row-Level Security: app.current_tenant_id]
    TAA --enforced by.-> RLS
    TAB --enforced by.-> RLS
    AGA --enforced by.-> RLS
    AGB --enforced by.-> RLS

    note1[Composite FKs carry tenant_id through every junction table,\nso a syntactically valid cross-tenant reference is\nrejected\nby the database itself, not only by application code.]
```

NEW IN v1.4 — Two More Closure Diagrams

5. Full Artifact Lineage Resolution (clauses 23, 59, 60, 61, 66 — closes CRITICAL “undefined run entities” finding)

```
flowchart LR
    OC[Owner Command] --> PR[project_registry]
    PR --> WF[workflow_registry]
    WF --> TR[task_registry]
    TR --> AR[agent_runs]
    AR --> AV[agent_versions]
    AR --> MR[model_runs]
    MR --> ART[artifact_registry]
    ART --> REV[Review: reviewed_by]
    REV --> APP[Approval: approved_by / approval_requests]
    APP --> GIT[git_commit_ref]
    GIT --> REL[release_registry]
    REL --> DEP[deployment_registry]
```

DEP --> MON[Monitoring]

note1[v1.3 defect: artifact_registry.agent_run_id / model_run_id\nreferenced tables that did not exist. v1.4 adds agent_runs\nand model_runs so this entire chain is now FK-resolvable.]

6. Authentication & Tenant Scoping Sequence (clauses 70, 71 — closes MEDIUM auth-contract finding)

```
sequenceDiagram
    participant C as Caller (human or agent/service)
    participant IdP as External Identity Provider
    participant API as Control Plane API
    participant RLS as Postgres RLS (app.current_tenant_id)

    C->>IdP: Authenticate (OAuth2 auth-code for humans, client-credentials for services)
    IdP-->>C: JWT (iss, aud, exp, tenant_id, scope, actor_type)
    C->>API: Request + Bearer JWT + Idempotency-Key
    API->>API: Validate signature against current JWKS, check iss/aud/exp (±60s skew)
    alt Token valid
        API->>RLS: SET app.current_tenant_id = JWT.tenant_id
        RLS-->>API: Session now scoped – USING + WITH CHECK enforced on every query
        API-->>C: Response (tenant-scoped result only)
    else Token invalid / expired
        API-->>C: 401 Unauthorized (ErrorResponse)
    end
    Note over IdP,API: Session/refresh-token lifecycle stays entirely with the IdP\n(clause 71) – the API only ever validates a presented token.
```

Section F — Agent Specification Template v1.4

Agent Specification Template v1.4

agent_id: tenant_id: # NULL only for a system-wide template; every executable agent must set this agent_name: department: role: purpose:

Capabilities

Required Model Capabilities

Tools (declared via MCP — Model Context Protocol, blueprint clause 54)

- tool_name: mcp_server: allowed_actions:

Permissions

Data Classification Allowed

- PUBLIC
- INTERNAL

Input Contract

{}

Output Contract

{}

Security Level

GREEN | YELLOW | RED

Sandbox Policy

Describe filesystem, network, database, shell and credential restrictions. Default sandbox credential lifetime: 15 minutes (AI provider keys), 24 hours (tool-gateway service credentials) — see blueprint clause 46.

Idempotency

Every task this agent executes must supply an idempotency_key; the agent must not perform a non-idempotent side effect (deployment, external write, message send) without one.

Evaluation Suite

- Accuracy
- Instruction following
- Tool precision
- Schema compliance
- Security
- Reliability
- Cost
- Latency
- Domain invariants

Lifecycle

DRAFT → SANDBOX → TESTED → EVALUATED → APPROVED → ACTIVE → UPDATED → DEPRECATED → RETIRED

Note (v1.3, blueprint clause 45): reaching APPROVED is the Agent Factory’s maximum authority. The APPROVED → ACTIVE transition requires a separate approval_requests record (action=AGENT_ACTIVATE) — the Factory pipeline cannot activate its own output into production.

Note (v1.4, blueprint clause 60): this specification produces a new, immutable row in agent_versions every time it changes — it never overwrites the agent’s currently ACTIVE version in place. Activation (above) targets a specific agent_version_id, so an agent may have version N active in production while version N+1 is still in SANDBOX/EVALUATED.

Tool Bindings (v1.4, blueprint clause 67)

Tools declared above are bound via agent_tool_bindings, keyed to a stable tool_id in the MCP tool_registry — not by free-text tool name — so Policy Engine and RBAC decisions resolve against a versioned, auditable tool identity.

External Interoperability (optional, disabled by default)

a2a_capability_card_enabled: false # blueprint clause 42/54 — only for # approved external partner integrations

Owner / Governance

Define who can approve activation and production permissions for this agent, scoped to tenant_id above.

Section G — Non-Functional Requirements & SLOs (All Items Labeled TARGET)

AI DIGITAL OFFICE OS v1.4 — Non-Functional Requirements & Service Level Objectives

This file was introduced in v1.3 because the v1.2 review found every non-functional requirement stated only as policy language (“bounded”, “auditable”, “scalable”) with no concrete number to test against. Every number below is a starting baseline meant to be tuned during Phase 1 load testing, not a marketing claim — treat these as the acceptance thresholds referenced in blueprint clause 53.

Evidence label (v1.4, blueprint clause 73): every number in this entire file is TARGET. None has been measured against a running system, because no running system exists yet (this remains a specification package — clause 74). When implementation testing produces real measurements, each TARGET below should be re-labeled TEST REQUIRED (a test exists but hasn’t run) or VERIFIED (a test ran and produced evidence) in the implementation-phase package — never edited to look VERIFIED in this specification document itself.

Availability

- Control plane API: target 99.9% monthly uptime (≈ 43 minutes allowed downtime/month).
- Agent execution layer: target 99.5% monthly uptime (agents may legitimately queue during provider failover; this is looser than the control plane by design).

Latency

- Control-plane read endpoints (`GET /tasks/{id}` , `GET /agents/{id}` , etc.): p95 < 300 ms, p99 < 800 ms.
- Task creation (`POST /tasks`) including Policy Engine evaluation: p95 < 1.2 s.
- Model routing decision (`POST /models/route`) excluding the downstream model call itself: p95 < 400 ms.
- Fallback trigger on primary provider failure: < 3 seconds from health-check failure to fallback provider invocation (blueprint clause 53).
- Memory semantic query (`POST /memory/query` , HNSW-indexed): p95 < 150 ms at up to 10M embedding rows per tenant shard.

Reliability — Retry & Backoff (blueprint clause 49)

- Retryable error classes: base delay 500 ms, multiplier 2x, jitter $\pm 20\%$, max 5 attempts, max total wait 60 s, then dead-letter/escalate.
- Non-retryable error classes (`VALIDATION_ERROR`, `AUTHORIZATION_ERROR`, `POLICY_ERROR`, `ARTIFACT_ERROR` immutability violations): escalate immediately, zero retries.
- All state-mutating operations require an idempotency key; duplicate delivery of the same key within a 24-hour window is a no-op returning the original result.

Scalability Targets (initial baseline, re-test each phase)

- 200 active agents per tenant (default policy ceiling, clause 5), horizontally scalable per tenant beyond that via explicit policy override.
- 10,000 concurrent tasks in `RUNNING/QUEUED` state across the platform before horizontal scale-out of the Task Engine is required.
- 1,000 tenants on shared schema before evaluating a shard-per-tenant-group split (the `shared-schema/tenant_id` pattern scales to tens of thousands of tenants per current practice; 1,000 is a conservative first re-evaluation point, not a hard ceiling).

Disaster Recovery (blueprint clause 50)

- Control-plane database: RPO $\leq$ 5 minutes, RTO $\leq$ 60 minutes.
- Artifact object storage: RPO $\leq$ 15 minutes, RTO $\leq$ 4 hours.
- Full restore drill: minimum quarterly, logged as a `decision_records` entry.

Security

- Credential lifetime: AI provider keys issued to a running task, 15 minutes; tool-gateway service credentials, 24 hours (clause 46).
- Cross-tenant isolation: 100% rejection rate on adversarial cross-tenant access test suite (minimum 50 cases), re-run on every schema migration (clause 53).
- Audit event retention: 400 days online, 5 years cold-archive minimum; RED-tier actions retained indefinitely (clause 57).

Cost Governance

- Per-tenant daily and monthly budget enforcement with three tiers: `WARNING` at 70% of budget, `SOFT_LIMIT` at 90% (routes to lower-cost models automatically where policy allows), `HARD_LIMIT` at 100% (blocks further spend pending Owner approval).

Observability

- Every request carries a `correlation_id` propagated across all services it touches (clause 29).
- Trace retention for agent decision paths (model chosen, alternatives rejected, policy result): minimum 90 days queryable (clause 55).

Self-Healing Bound

- Maximum 3 automatic self-healing attempts per failure before mandatory human escalation (clause 32, unchanged from v1.1/v1.2).

Section H — Implementation Acceptance Checklist v1.4 (Evidence-Disciplined)

AI DIGITAL OFFICE OS v1.4 Implementation Acceptance Checklist

Evidence-Disciplined Version (blueprint clause 73)

Rule for this entire file: every item below carries exactly one label — **TARGET** (design goal, not yet tested), **TEST REQUIRED** (a test exists or is planned but has not run), or **VERIFIED** (an executed test produced recorded evidence — test run ID, timestamp, result). No item may be marked VERIFIED on the strength of this specification's text alone. As of this document's writing, **every item is TARGET** because no running implementation exists yet (clause 74) — this file is the checklist to run against, not a record of what has already passed.

Architecture — [TARGET]

- v1.1/v1.2/v1.3 architecture preserved — all 9 layers present and connected
- Agent Factory integrated, stops at APPROVED (cannot self-activate — clause 45)
- Provider Gateway integrated with at least 2 live provider adapters passing health checks
- Model Registry integrated, router fallback verified to trigger in < 3 seconds (nfr_slo_v1.4.md)
- Policy Engine integrated, returns ALLOW/DENY/REQUIRE_APPROVAL/REQUIRE_ESCALATION on 100% of evaluated test cases
- Approval Engine integrated with all 3 risk tiers correctly routed

Project Registry — [TARGET] (new in v1.4, closes CRITICAL finding)

- `project_registry` exists and every operational `project_id` (task, workflow, artifact, decision, deployment, release) resolves to it via FK, with zero orphaned references
- Project lifecycle endpoints (`/projects` , `/projects/{id}`) function end-to-end

Agent Versioning — [TARGET] (new in v1.4, closes CRITICAL finding)

- `agent_versions` stores every version as an immutable row; updating an ACTIVE agent never overwrites the existing version row
- Activation targets a specific `agent_version_id` ; an agent can have version N ACTIVE while version N+1 is still SANDBOX/EVALUATED
- `POST /agents/{id}/versions` and `.../activate` function end-to-end

Lineage — [TARGET] (new in v1.4, closes CRITICAL finding)

- `agent_runs` and `model_runs` exist; `artifact_registry.agent_run_id / model_run_id` resolve via FK with zero unresolved references
- `GET /artifacts/{id}/lineage` reconstructs Owner Command → Project → Workflow → Task → Agent Run → Model Run → Artifact → Review → Approval → Release → Deployment for a sampled artifact with no gaps

Multi-Tenancy & RLS — [TARGET] (full coverage new in v1.4, closes CRITICAL finding)

- RLS is enabled with both `USING` and `WITH CHECK` on every tenant-scoped operational table (not only the 4-table v1.3 baseline) — see clause 62 for the full table list
- Adversarial cross-tenant access test suite (≥ 50 cases, covering read AND write attempts) passes at 100% rejection
- System-template NULL-tenant rows verified constrained to template-only tables (`agent_registry`, `roles`, `mcp_server_registry`, `feature_flags`)

Agent Messaging — [TARGET] (idempotency added in v1.4, closes HIGH finding)

- `agent_messages.idempotency_key + UNIQUE(tenant_id, idempotency_key)` enforced; replaying the same key returns the original result, verified with duplicate-delivery test
- A2A external interoperability confirmed disabled by default

AI Platform — [TARGET]

- Provider Registry, adapters, health probes operational
- `model_evaluation_runs + model_evaluation_metrics` versioned and rerunnable (new in v1.4, closes HIGH finding) — re-evaluating a model creates a new row, never overwrites history
- Intelligent routing decisions 100% auditable via `routing_decision_records` (new in v1.4, closes MEDIUM finding)
- Policy decisions 100% auditable via `policy_decision_records` (new in v1.4, closes MEDIUM finding), including alternatives considered and rejection reasons
- Fallback chain tested under simulated provider outage

Cost & Usage — [TARGET] (new in v1.4, closes HIGH finding)

- `usage_events` ledger populated for every billable unit of work; reconciles against `/costs` aggregation with zero discrepancy in a sampled period
- WARNING/SOFT_LIMIT/HARD_LIMIT budget tiers demonstrably trigger at 70%/90%/100%

Deployment & Release — [TARGET] (new in v1.4, closes HIGH finding)

- `release_registry` and `deployment_registry` replace the free-text `deployment_ref`; every deployment has a resolvable `rollback_target`
- Tested rollback completes within the RTO target for the affected component (`nfr_slo_v1.4.md`)

MCP Tool Registry — [TARGET] (new in v1.4, closes HIGH finding)

- `mcp_server_registry` and `tool_registry` are authoritative; `agent_tool_bindings` is tenant-scoped and policy-checkable
- A disabled/untrusted MCP server or tool cannot be invoked by any agent, verified by test

API Completeness — [TARGET] (expanded in v1.4, closes HIGH finding)

- All 40 `/v1` endpoints in `control_plane_openapi_v1.4.yaml` respond per their documented schema, including the v1.4 additions: projects, workflow cancel/retry/escalate, agent version create/list/activate, agent message send, artifact create/lineage, model evaluation trigger/history, tools/MCP listing, usage/costs, deployment create/get/rollback, policy/routing decision queries
- Every state-changing endpoint enforces the `Idempotency-Key` header (clause 68)

Schema/API Consistency — [TARGET] (new in v1.4, closes HIGH finding)

- A CI schema-diff test (or canonical-domain-model code generation) fails the build on any SQL-column vs. OpenAPI-field drift, verified by intentionally introducing a drift in a test branch and confirming the gate catches it

Authentication & Session Boundary — [TARGET] (detailed in v1.4, closes MEDIUM finding)

- JWT validation enforces `iss`, `aud`, `exp` / `iat` (max 15 min service / 8 hr human), `tenant_id`, `scope`, `actor_type`, with $\pm 60s$ clock skew
- JWKS rotation tested: old and new keys both validate during the 7-day overlap window
- Session/refresh-token lifecycle confirmed delegated to the external IdP with no in-house session store (clause 71) — or, if that decision is revisited, a new `session_registry` design is reviewed separately before being added

Security — [TARGET]

- RBAC/ABAC enforced at both gateway and service layer
- Least privilege: a role with zero permissions can perform zero actions
- Secrets vault: automated scan finds zero raw secret values in the application database
- Audit trail reconstructable end-to-end for a sampled artifact with zero `correlation_id` gaps

Engineering — [TARGET]

- Task engine: all 11 states reachable and tested
- Durable workflows: forced process restart mid-workflow resumes from `workflow_history` with zero task duplication
- Dependency DAG: circular dependency/deadlock detection verified with synthetic cases
- CI/CD: unit, integration, static analysis, security scan all gate merge; zero unresolved CRITICAL findings before production

Memory & Artifacts — [TARGET]

- Working memory TTL enforced (default 4h); durable memory facts queryable by scope
- Semantic memory (pgvector/HNSW) query p95 < 150 ms at target scale
- `embedding_model` column enforced — mixed-model embedding comparison blocked
- Artifact immutability: an attempted mutation of an APPROVED artifact creates a new `artifact_id` instead of overwriting

Operations — [TARGET]

- Logs, metrics, traces, events, alerts cover all 9 layers plus the v1.4 lineage/ledger tables
- Agent-specific trace events capture rejected alternatives (model routing, policy decisions)
- Disaster recovery: $RPO \leq 5 \text{ min}$ / $RTO \leq 60 \text{ min}$ (database), $RPO \leq 15 \text{ min}$ / $RTO \leq 4 \text{ hr}$ (artifacts) demonstrated in a drill

Extensibility — [TARGET]

- Add provider/model/agent/tool/project-type without core schema/API rewrite, each demonstrated with a concrete example
- Add tenant without any schema migration (pure data insert into `organizations`)

Product Independence — [TARGET] (new in v1.4, user-directed change)

- No schema column, API field, or agent template found to be specific to any single named downstream product, verified by review of every table and endpoint against clause 75
- Confirm no lingering reference to the previously-named coupled product remains anywhere in blueprint, schema, API, diagrams, or checklist

Evidence Discipline — [TARGET] (new in v1.4, closes MEDIUM finding — this rule governs this whole file)

- No checklist item anywhere in this document is marked VERIFIED without a corresponding executed test producing recorded evidence
- SLO numbers in `nfr_slo_v1.4.md` are measured only after a running environment exists
- Production security claims are backed by executed tests, not specification text

Implementation-Lock Gate — [TARGET] (blueprint clause 76)

- All five gate conditions in clause 76 hold with VERIFIED evidence: schema/API/domain-model consistency, full RLS with adversarial test pass, end-to-end lineage resolution, immutable version-targeted activation, and every checklist item correctly labeled

Section I — Changelog: v1.3 → v1.4 (Finding-by-Finding Closure Map)

Changelog — v1.3 → v1.4

Why this version exists

An independent Technical Integrity Review of the v1.3 Consolidated Master (dated 2026-08-30) found v1.3 “substantially stronger than v1.2” and architecturally correct, but identified specific structural closure issues between the blueprint narrative, the SQL schema, and the OpenAPI contract. v1.4 closes every finding in that review, one-for-one. Nothing in v1.3 was removed or weakened; v1.4 only adds — with one deliberate exception (clause 34) made on explicit user instruction, documented separately below.

CRITICAL findings — closed

Review finding	v1.4 closure
Project Registry missing	Added <code>project_registry</code> ; retrofitted FKs from <code>task_registry</code> , <code>workflow_registry</code> , <code>artifact_registry</code> , <code>decision_records</code>
Agent versioning structurally incomplete	Added <code>agent_versions</code> (immutable per-version rows); <code>agent_registry.active_agent_version_id</code> now points at the live version; activation targets a version, not just an agent
Artifact lineage references undefined run entities	Added <code>agent_runs</code> and <code>model_runs</code> ; retrofitted real FKs from <code>artifact_registry.agent_run_id</code> / <code>model_run_id</code> , which were unbacked VARCHAR columns in v1.3
RLS only partially implemented	Extended RLS from 4 tables (USING only) to every tenant-scoped operational table (23+ tables), each with both <code>USING</code> and <code>WITH CHECK</code>

HIGH findings — closed

Review finding	v1.4 closure
Message idempotency not stored	Added <code>agent_messages.idempotency_key</code> + <code>UNIQUE(tenant_id, idempotency_key)</code>
Model evaluation history missing	Added <code>model_evaluation_runs</code> + <code>model_evaluation_metrics</code> ; <code>model_registry.evaluation_score</code> is now a denormalized “latest” convenience field only
Cost/usage ledger missing	Added <code>usage_events</code> as the authoritative ledger
Deployment/release records missing	Added <code>deployment_registry</code> + <code>release_registry</code> ; <code>artifact_registry.deployment_ref</code> marked deprecated in the API

	schema
MCP tool registry missing	Added <code>mcp_server_registry</code> , <code>tool_registry</code> , <code>agent_tool_bindings</code>
OpenAPI incomplete lifecycle coverage	Expanded from 18 to 40 endpoints — projects, workflow cancel/retry/escalate, agent version create/list/activate, agent message send, artifact create/lineage, model evaluation trigger/history, tools/MCP listing, usage/costs, deployment create/get/rollback, policy/routing decision queries
OpenAPI schemas narrower than SQL	<code>AgentRecord</code> , <code>Artifact</code> , <code>FeatureFlag</code> schemas expanded to match their SQL tables field-for-field

MEDIUM findings — closed

Review finding	v1.4 closure
Auth contract underspecified	Documented <code>iss / aud / exp / iat / scope / actor_type</code> claims, JWKS rotation (90-day cycle, 7-day overlap), ± 60 s clock skew, token lifetimes (15 min service / 8 hr human)
Session management unmodeled	Explicitly delegated to the external IdP (decision made, not left ambiguous) — documented as a clause and as a boundary note in the OpenAPI security scheme
Policy/routing decisions not first-class	Added <code>policy_decision_records</code> and <code>routing_decision_records</code>
Acceptance claims blur target vs. measured	Every checklist item now carries TARGET / TEST REQUIRED / VERIFIED; all currently TARGET
Master PDF is spec, not implementation	Reaffirmed explicitly (clause 74); recommended a separate implementation-phase package for IaC/credentials/real adapters, not built here

Also in this version — user-directed product decoupling

On explicit instruction, all coupling between AI DIGITAL OFFICE OS and any specific named downstream product has been removed: - Blueprint clause 34 rewritten from naming a specific POS project as the “first major factory implementation” to a fully generic, product-agnostic statement: any project a tenant builds is an ordinary `project_registry` row with zero special-cased agents, tables, or bypass paths. - The one Layer-5 mention of “locked [product] operational agents” changed to “tenant-defined domain operational agents.” - Blueprint clause 38, Phase 8, changed from naming a specific product launch to “first production project launch (product-agnostic).” - New clause 75 formalizes this as a standing architectural principle. - This is a deliberate content edit to clause 34, breaking the general “preserve 1–40 verbatim” rule that has held since v1.3 — done once, intentionally, and only for this clause, on direct user request.

What did NOT change

- The approved hierarchy, Agent Factory, capability-based model routing, multi-provider gateway, Policy Engine, GREEN/YELLOW/RED governance, durable workflows, tiered memory, artifact immutability, independent QA, bounded self-healing, and MCP/A2A direction — unchanged, per the review’s own “what must not change” section.
- All v1.3 NFR/SLO numbers — unchanged, now explicitly re-labeled TARGET.

Explicitly NOT done (honesty — clause 74)

- No infrastructure, cloud topology, provider contracts, pricing, penetration-test results, or production credentials were added.
- No code implementation was produced. This remains a specification and data-model package. The review’s own recommendation — a separate implementation-phase package containing IaC, deployment topology, real provider adapters, and executed test evidence — is a distinct future deliverable, not part of v1.4.
- The five conditions of the Implementation-Lock Gate (blueprint clause 76) are stated but not yet satisfied with VERIFIED evidence, because no running system exists to test against yet.

Section J — Baseline & Final Honesty Statement

The complete v1.3 package (blueprint, schema, API, diagrams, template, NFR doc, checklist, changelog, and the 31-page v1.3 Consolidated Master PDF) is preserved unmodified inside this package’s `v1.3_baseline/` folder. Nothing in v1.3 was deleted; v1.4 only closes the structural findings an independent technical integrity review raised against it.

This v1.4 package is now internally consistent against that review: every CRITICAL finding (Project Registry, agent versioning, artifact/run lineage, full RLS) and every HIGH and MEDIUM finding (message idempotency, model evaluation history, cost/usage ledger, deployment/release records, MCP tool registry, complete API lifecycle, schema/API consistency, detailed auth contract, session boundary, policy/routing decision records, evidence discipline) has a corresponding table, endpoint, or

documented decision. On explicit user instruction, the specification is also now fully decoupled from any specific named downstream product — see blueprint clauses 34 and 75.

Per the blueprint's own clause 76 (Implementation-Lock Gate): this remains an **implementation-lock candidate**, not yet implementation-locked. It becomes lockable only once the five gate conditions — schema/API/domain-model consistency, full RLS with a passing adversarial test, end-to-end artifact lineage resolution, immutable version-targeted agent activation, and every checklist item correctly evidence-labeled — are satisfied with **VERIFIED** evidence from an actual test run, not specification text alone. No infrastructure-as-code, cloud topology, real provider contracts, penetration-test results, or production credentials are included or fabricated here; those belong to a separate, future implementation-phase package.